

 **DTU Compute**
Department of Applied Mathematics and Computer Science

Stabilizing Gradient Inversion in Federated Learning

Alfred Aqraou (s234820)

Ali Afif (s234832)

Mathias Kræmer Eberhardt Sørensen (s234843)



June, 2026

Stabilizing Gradient Inversion In Federated Learning

Bachelor of Science in Engineering

BSc. Eng. Thesis

June, 2026

By

Alfred Agraou, Ali Afif, Mathias Kræmer Eberhardt Sørensen

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Supervisor: Hilba Nassar

DTU Compute
Department of Applied Mathematics and Computer Science
Technical University of Denmark
Building 324
2800 Kongens Lyngby, Denmark

Preface

This thesis has been prepared over 18 weeks at DTU Compute, Department of Applied Mathematics and Computer Science, at the Technical University of Denmark, DTU, in partial fulfillment of the requirements for the degree Bachelor of Science in Engineering, BSc. Eng.

The reader is assumed to have a working knowledge of machine learning and linear algebra. Familiarity with neural network training and gradient-based optimization is beneficial but not required.

This thesis was developed collaboratively using the Git version control system. Details regarding the codebase can be found in Appendix A.

Generative AI tools were used to assist with proofreading, code development, and research; all output was reviewed and verified by the authors, who take full responsibility for this work.

Alfred Aqraou

Stud. BSc Eng. (Artificial Intelligence and Data) Stud. BSc Eng. (Artificial Intelligence and Data)
Technical University of Denmark (DTU) Technical University of Denmark (DTU)
s234820@dtu.dk s234832@dtu.dk



Signature



Signature

Mathias Kræmer Eberhardt Sørensen

Stud. BSc Eng. (Artificial Intelligence and Data)
Technical University of Denmark (DTU)
s234843@dtu.dk



Signature

Abstract

Federated learning trains shared models without exchanging training data, relying on the assumption that gradient updates reveal less than the inputs that produced them. Gradient inversion attacks challenge this assumption by reconstructing training images from shared gradients. However, such attacks remain unreliable in practice, particularly for deeper architectures, even when the gradients may still retain sufficient information for reconstruction. This raises a central question for the threat model: when reconstruction fails, is the information absent from the gradients, or is it present but difficult to extract through optimization?

This thesis investigates that question empirically by selectively removing components from the gradient before inversion and testing whether the reduced subset produces better reconstructions than the full gradient. Experiments cover LeNet, VGG, and ResNet architectures with both randomly initialized and pretrained weights. The most striking result is VGG-11, where masking the classifier and first feature layer under Signed AdamW achieves near-perfect reconstruction: mean SSIM rises from 0.62 to 0.94 (+52%), and mean MSE falls by 94%. Extended analysis on VGG-13, a deeper and harder-to-invert network, confirms the pattern: mean PSNR increases from 15.81 to 23.14 dB on CIFAR-100 (+99% SSIM) and from 14.14 to 22.50 dB on LFW facial images (+137% SSIM, 35 of 50 images reconstructed), demonstrating that the full gradient is not the most effective reconstruction signal and that recoverable information can remain hidden from baseline attacks.

The benefit is not universal. Masking is most effective when disruptive gradient components are concentrated in identifiable layers, as observed for sequential VGG models, but provides no consistent benefit for ResNet architectures, where gradient information is distributed through residual connections. Pretraining reverses this pattern for sequential architectures: layers that improve reconstruction when masked in randomly initialized networks instead degrade it once the network carries learned weights. Numerical rank estimates confirm that the retained subsets still satisfy the structural conditions for input recovery. Restart experiments further show that the improvement from masking is not explained by a favorable initialization alone. These results show that poor reconstruction does not by itself imply that gradients lack recoverable information; in some settings, selective masking exposes privacy risks that baseline inversion leaves hidden.

Acknowledgements

We would like to thank our supervisor Hiba Nassar for her support through the project.

Contents

1	Introduction	1
2	Background and Related Work	3
2.1	Federated Learning	3
2.2	Related works	5
3	Data	7
3.1	CIFAR-100	7
3.2	Labeled Faces in the Wild (LFW)	7
4	Theory	8
4.1	DLG	8
4.2	iDLG	9
4.3	Networks	10
4.4	Pretrained Networks	12
4.5	Masking	13
4.6	Non-Convex Optimization	16
4.7	Gradient Matching Loss Functions	17
4.8	Optimizers	18
4.9	Activation functions	20
4.10	Evaluation metrics	21
5	Methods	23
5.1	Verifying necessary conditions for image reconstruction	23
5.2	Gradient Inversion Attacks Implementation	24
5.3	Implementation of Evaluation Metrics	27
5.4	Paired statistical comparison of reconstruction quality	28
6	Results	29
6.1	Normality tests	29
6.2	LeNet	29
6.3	VGG	30
6.4	ResNet	34
6.5	Jacobian Rank	36
6.6	Number of images reconstructed vs masking percentage	37
6.7	Effect of Restarts	38
7	Discussion	41
7.1	Interpretation of LeNet results	41
7.2	Interpretation of VGG results	42
7.3	Interpretation of ResNet results	43
7.4	Synthesis	44
7.5	Limitations and future work	45
8	Conclusion	47

Bibliography	49
A Code documentation	51
B Jacobian Rank	52
B.1 Rank Reconstruction Sweep	52
C Ablation studies	54
C.1 LeNet	54
C.2 ResNet	55
C.3 VGG	58
D Additional pictures	60
D.1 VGG-11	60
D.2 VGG-13 best masking config	61
D.3 LeNet	62
D.4 ResNet	62
E Model Parameter Counts	63

Introduction

The continued advancement of machine learning relies on the availability of training data. Each gain in model capability from medical diagnostics to financial risk assessment, has followed from training on progressively larger and more representative datasets, and sustained progress demands an ever greater supply. Historically, that need was met by centralizing the data: pooling datasets from many sources into shared repositories that training pipelines could draw on freely.

Federated learning (FL) was proposed as a framework for training under these constraints [1]. Rather than centralizing data, clients compute gradient updates on their own local datasets and transmit only those updates to a server, which aggregates them into a shared model. No training data crosses local devices. The framework has moved well beyond a research prototype: diagnostic models are trained across hospital networks, fraud-detection signals are shared between financial institutions, and on-device models are improved without routing user data to a server. In each case, the privacy guarantee offered to users and regulators rests on the same premise: sharing gradients is fundamentally different from sharing inputs, because gradient vectors carry no recoverable information about the data that produced them.

That premise did not survive investigation.

A line of research beginning with Zhu et al. [2] showed that training inputs can be reconstructed directly from gradient updates. For small networks and single training samples the reconstruction can be nearly exact, recovering images that are visually indistinguishable from the originals. Subsequent work derived analytical methods to recover labels directly from the gradient [3] and extended the attack to deeper networks and ImageNet-scale images through improved optimization objectives and image priors [4]. Across this body of work, each paper expanded what an adversary could recover from a single gradient update.

In practice, these attacks exhibit limited reliability, and their reported success conditions rarely resemble real federated learning deployments. The most frequently cited demonstrations assume a single training sample, a shallow network trained from a random initialization, and an adversary with complete knowledge of the model architecture. Remove any of these conditions and performance degrades sharply. As the local batch size increases from one to just a handful of samples, the gradient becomes a sum of contributions from multiple inputs, and disentangling them grows increasingly difficult. Deeper architectures produce gradient landscapes that are harder to invert, and networks with pretrained weights carry structured semantic representations that appear to obscure low-level pixel information. In more realistic settings, methods that achieve compelling results on controlled benchmarks produce noise or semantically incoherent reconstructions [5]. Whether this reflects a fundamental limit on recoverable information or a failure of the optimization is not immediately clear from the reconstruction alone.

This thesis argues that many gradient inversion failures arise from numerical instability rather than a genuine absence of recoverable information. Gradient inversion is a non-convex optimization problem over a high-dimensional search space, with no convergence guarantees and a loss surface populated by local minima and saddle points. Although a full gradient contains contributions from every layer of a neural network, these components do not carry equal information about the original input. When less informative gradients dominate the inversion objective, the optimization signal can degrade even

when reconstructive information remains present elsewhere. These failures are observationally indistinguishable from cases where gradients genuinely lack reconstructive information, but the distinction has important implications. If gradients genuinely lack recoverable information, gradient sharing is inherently safe. If failures instead stem from optimization difficulties, privacy depends on the current limitations of adversarial methods, and improvements in optimization or computational resources may increase reconstruction success.

To investigate this distinction, this thesis proposes an empirically guided gradient masking strategy that selectively removes gradient components from the inversion objective before optimization. By retaining informative components while discarding those that hinder convergence, the approach aims to stabilize the inversion process without sacrificing the information required for reconstruction. The central question is whether such stabilization can make gradient inversion more reliable in settings where existing methods fail, and what the resulting patterns of improvement reveal about how reconstructive information is distributed across different neural network architectures.

Background and Related Work

2.1 Federated Learning

Rather than pooling training data on a central server, federated learning distributes the optimization problem across K clients, each holding a private local dataset of size n_k . Proposed by McMahan et al. [1], the framework coordinates training through iterative rounds in which clients compute local gradient updates and transmit them to a server for aggregation. The shared objective is to minimize a weighted average of local empirical risks:

$$\min_w F(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w), \quad (2.1)$$

where $F_k(w)$ denotes the average training loss computed over the local dataset at client k , n_k is the size of the local dataset at client k , and $n = \sum_{k=1}^K n_k$ is the total number of training samples across all participating clients.

FedSGD

The simplest instantiation of this framework is Federated SGD (FedSGD). In each communication round, the server distributes the current global model to a selected subset of clients. Each client computes a single full-batch gradient over its local data and returns it to the server, which aggregates them as a weighted average:

$$w^{t+1} \leftarrow w^t - \eta \sum_{k=1}^K \frac{n_k}{n} \nabla F_k(w^t). \quad (2.2)$$

Here w^t denotes the global model parameters at round t , η is the learning rate, and $\nabla F_k(w^t)$ is the full-batch gradient computed by client k over its local dataset. While conceptually straightforward, FedSGD requires one round of server communication per gradient step, making it impractical for real deployments.

FedAvg

To address this, McMahan et al. proposed Federated Averaging (FedAvg) [1], which introduces three control parameters: C , the fraction of clients selected per round; E , the number of local training epochs; and B , the local mini-batch size. FedSGD corresponds to the special case $E = 1$, $B = \infty$. In FedAvg, each selected client instead performs E epochs of mini-batch SGD on its local objective before communicating. Rather than sending raw gradients, clients return their updated model parameters w_k^{t+1} , and the server aggregates these through weighted averaging:

$$w^{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_k^{t+1}. \quad (2.3)$$

This reduces the number of required communication rounds substantially, with reductions of 50–100× reported in the original work. However, allowing multiple local updates introduces additional

challenges when client data is non-independent and identically distributed (non-IID): clients hold data drawn from different underlying distributions, exhibiting variation in label frequencies, feature distributions, and dataset sizes. Performing multiple local optimization steps amplifies *client drift*, as each client increasingly optimizes toward its own local objective $F_k(w)$ rather than the global objective $F(w)$, complicating convergence and degrading performance relative to centralized training.

Although federated learning reduces exposure to raw data, it does not eliminate the underlying privacy risk. The gradient updates transmitted during training still encode information about the client’s local data, and, as established in the introduction, this information may be sufficient to reconstruct the original inputs. Yet, the presence of information in the gradients does not guarantee successful recovery in practice: reconstruction success may depend not only on whether information exists, but also on whether the available gradient entries yield a well-conditioned optimization problem. This work therefore asks whether that gap reflects a true lack of recoverable information, or whether the information is present but difficult for the optimization to exploit, a question investigated through gradient masking.

Scope of This Work. The experiments in this thesis adopt the standard threat model assumed in the gradient inversion literature [2, 3, 4], focusing on the FedSGD setting in which each client performs a single forward-backward pass over a private training sample and shares the resulting gradient directly with the server, as illustrated in Figure 2.1.

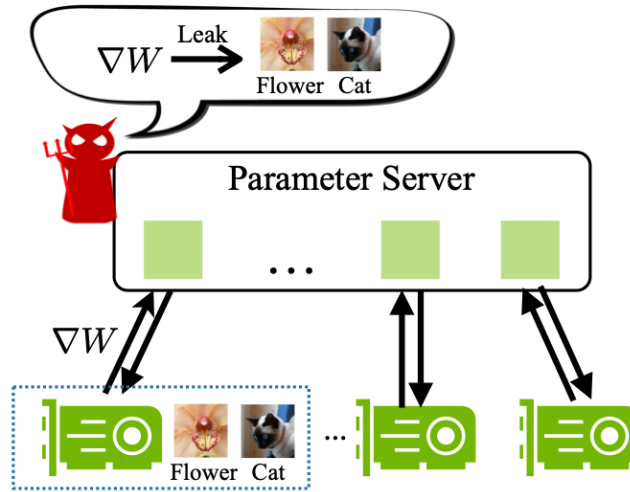


Figure 2.1. Illustration of the distributed training setup assumed in this work. Worker nodes compute gradients locally on private training data and transmit them to a central parameter server for aggregation and model updates. Because these gradients are shared during normal training, an adversarial server (illustrated by the red demon) may exploit them to reconstruct sensitive training samples [2].

Formally, at each step t , a client computes

$$\nabla W_t = \frac{\partial \ell(F(x_t, W_t), y_t)}{\partial W_t}, \quad (2.4)$$

and transmits ∇W_t , the gradient of the loss with respect to the model parameters, to the server. Given this shared gradient, an attacker with knowledge of the model F and its current parameters W_t aims to recover the private training sample (x_t, y_t) .

No full federated training system is implemented. The FL framework serves as the motivation and threat model: it defines who shares what with whom, and under what assumption privacy was expected to hold. As the number of local epochs increases, accumulated weight updates increasingly obscure the per-sample gradient signal, making reconstruction harder. The single-step setting therefore represents

the most favorable condition for an attacker and is the standard evaluation protocol in the gradient inversion literature.

2.2 Related works

2.2.1 DLG

Deep Leakage from Gradients (DLG), introduced by Zhu et al. [2], is one of the first studies to show that gradients shared during training can contain enough information to reconstruct private training data. This challenged a key assumption in federated learning: that privacy is preserved as long as the data remains on the local device. The authors demonstrated that the gradients themselves may reveal sensitive information about the samples used to compute them.

The attack is based on gradient inversion. The attacker initializes dummy data and labels at random, computes their gradients under the same model, and iteratively updates them to minimize the difference from the observed gradients. If successful, the optimized dummy data becomes a reconstruction of the original training sample. The mathematical details of this optimization are presented later in the theory section.

Zhu et al. evaluated DLG on several image datasets, including MNIST, CIFAR-100, SVHN, and LFW, using a modified ResNet-56 architecture. Their results showed that recognizable images could be reconstructed from shared gradients, with lower Mean Squared Error (MSE) than previous reconstruction methods. Importantly, the study showed that even a single gradient update may be enough to leak private information, making DLG an important baseline for later research on gradient inversion attacks.

2.2.2 iDLG

Improved Deep Leakage from Gradients (iDLG), proposed by Zhao et al. [3], extends DLG by addressing one of its main limitations: unreliable label recovery. In the original DLG method, both the dummy input and the dummy label are optimized. This can make the reconstruction process less stable, since an incorrect label can directly affect the quality of the recovered input.

The main contribution of iDLG is to show that the ground-truth label can be recovered directly from the shared gradients under common training conditions, specifically when using softmax and cross-entropy loss. Instead of optimizing a dummy label, iDLG identifies the true label from the sign pattern of the final-layer gradients. This simplifies the reconstruction problem and allows the attack to focus on optimizing only the dummy input. The theoretical reasoning behind this label recovery method is described in more detail in the following chapter.

Zhao et al. evaluated iDLG against DLG on MNIST, CIFAR-100, and LFW, using a randomly initialized LeNet-style convolutional network with sigmoid activations and the L-BFGS optimizer. Across 1,000 experiments for each dataset, iDLG achieved 100% label recovery accuracy, while DLG recovered labels with lower accuracy, reported between 80–90%. In addition, iDLG produced better image reconstructions, measured by lower MSE. These results show that reliable label recovery is crucial for successful gradient inversion, and that iDLG makes DLG more stable by recovering the ground-truth label analytically. Nevertheless, this improvement does not address the optimization of the dummy image itself, which remains a limiting factor for deeper architectures and more complex data.

2.2.3 Inverting Gradients

Geiping et al. [4] showed that the limited success of earlier attacks such as DLG and iDLG was largely due to weak optimization in their paper *Inverting Gradients – How easy is it to break privacy in federated learning?*. They argue that Euclidean gradient matching with L-BFGS is numerically unstable for deep networks using ReLU activations, instead they propose a cosine-similarity loss function that focuses on matching gradient direction rather than magnitude, combined with the Adam optimizer using step size decay, making gradient inversion substantially more stable and effective. Lastly, the images are constrained to be within $[0, 1]$ and total variation (TV) is added as a simple image prior, leading to the minimization of equation (2.5)

$$\arg \min_{x \in [0,1]^n} 1 - \frac{\langle \nabla_{\theta} \mathcal{L}_{\theta}(x, y), \nabla_{\theta} \mathcal{L}_{\theta}(x^*, y) \rangle}{\|\nabla_{\theta} \mathcal{L}_{\theta}(x, y)\| \|\nabla_{\theta} \mathcal{L}_{\theta}(x^*, y)\|} + \alpha \text{TV}(x). \quad (2.5)$$

This yields significant improvements in reconstruction quality on deeper and trained networks due to improved numerical stability [4].

The paper also theoretically proves that the input to a fully connected network (FCN) can be uniquely reconstructed from the network’s gradients. Experimentally, the proposed method substantially improves reconstruction quality on trained and untrained ResNet models, but is outperformed on simple untrained networks, as shown in Table 2.1.

Architecture	LeNet (Zhu)		ResNet20-4	
	False	True	False	True
Eucl. Loss + L-BFGS	46.25 ± 12.66	13.24 ± 5.44	10.29 ± 5.38	6.90 ± 2.80
Proposed	18.00 ± 3.33	18.08 ± 4.27	19.83 ± 2.96	13.95 ± 3.38

Table 2.1. PSNR mean and standard deviation for 100 experiments on the first images of the CIFAR-10 validation data set over two different networks with trained and untrained parameters. **Bold** indicates the best performing method per column. [4]

The authors further demonstrate recognizable reconstructions for large models such as ResNet-152 and show that federated averaging over multiple images remains vulnerable to gradient inversion attacks (GIAs). Among the defenses evaluated, they conclude that differential privacy is the only mechanism that provides a formal guarantee against reconstruction [4]. These empirical findings motivate a more theoretical question: under what structural conditions do shared gradients contain enough information to make reconstruction possible in the first place?

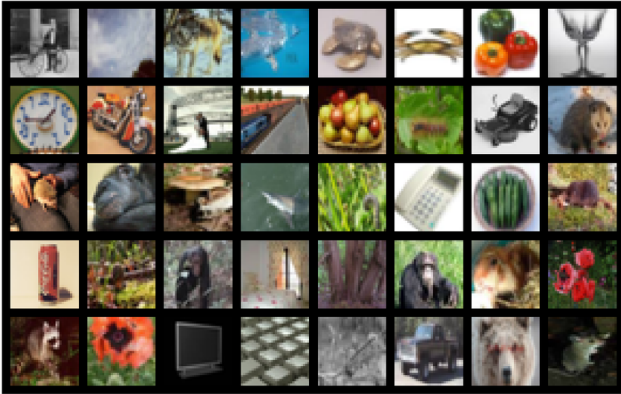
2.2.4 Minimal Model Structure for Input Reconstruction

Qian et al. [5] address this by framing gradient based input reconstruction as solving a system of linear equations, deriving minimal architectural requirements for when reconstruction is guaranteed. For MLPs, the number of neurons in the first hidden layer must be at least as large as the batch size; for CNNs, a related width condition applies to the number of convolutional kernels. With the precise bound determined by the input dimension d , output dimension after convolution d' , and number of input channels C : $h \geq (\frac{d}{d'})^2 C$. These bounds treat each gradient equation as an independent constraint on the unknown input.

This chapter describes the image datasets used to evaluate gradient inversion and gradient masking. The datasets were chosen to represent realistic federated learning scenarios, spanning varying levels of visual complexity and covering the kind of sensitive data, object categories and facial images, that clients would potentially want to keep private.

3.1 CIFAR-100

The CIFAR-100 dataset comprises a total of 60,000 32×32 RGB natural images. Each image is represented as a tensor of dimension $32 \times 32 \times 3$. The dataset contains 100 specific object classes that are hierarchically grouped into 20 broader superclasses, meaning each superclass encompasses exactly 5 distinct, fine-grained classes. With exactly 500 training images and 100 test images per specific class, the dataset is perfectly balanced but significantly more granular. The dataset is loaded using the torchvision implementation. Images are normalized using per-channel means and standard deviations computed from the respective training sets, following the standard values established in the inverting gradients literature for CIFAR-100, $\mu = (0.507, 0.487, 0.441)$ and $\sigma = (0.267, 0.256, 0.276)$ [4].



(a) CIFAR-100 Dataset [6]



(b) LFW dataset [7]

Figure 3.1. Sample images of the datasets

3.2 Labeled Faces in the Wild (LFW)

LFW [8] is a dataset of 13,233 images consisting of 5,749 face identities with varying poses and lighting conditions. Images are resized to 32×32 RGB and represented as tensors of dimension $32 \times 32 \times 3$. Unlike CIFAR-100, LFW is highly imbalanced, with many identities represented by only a few images while a small number of individuals have a large number of samples. The dataset is obtained from the Kaggle distribution [9], which contains all 13,233 images, and is processed through a custom dataset wrapper that aligns it with the input format used for the CIFAR-100 dataset. This dataset is particularly important for evaluating privacy risks in sensitive biometric applications where facial data is stored locally on clients. Testing gradient inversion on face recognition tasks demonstrates vulnerabilities in federated scenarios involving personal biometric information, where class imbalance reflects real-world conditions.

Answering whether reconstruction failure reflects absent information or optimization difficulty requires understanding both what gradients encode and how the inversion problem is formally structured. This chapter establishes that theoretical basis, treating gradient inversion as an inverse optimization problem in which a dummy input is optimized until its gradients match those observed from the true training example; it introduces the DLG and iDLG attacks, the matching objectives and optimizers used to solve it, and the architectural properties that determine how information is represented in gradients. The final section formalizes the role of masking through rank-based reconstruction conditions and defines the image-quality metrics used throughout the evaluation.

4.1 DLG

Deep Leakage from Gradients (DLG) [2], introduced in 2019, was the first method to show that private training data can be reconstructed from shared gradients with pixel-level accuracy. Given the gradient $\nabla W_{t,k}$ shared by participant k at step t , where W denotes the model parameters, the attack recovers the training sample $(x_{t,k}, y_{t,k})$ without any access to the private data.

DLG initializes a dummy input $x' \leftarrow \mathcal{N}(0, 1)$ and label $y' \leftarrow \mathcal{N}(0, 1)$ and computes the induced dummy gradient $\nabla W' = \frac{\partial \ell(F(x', W), y')}{\partial W}$. Reconstruction is then posed as finding the dummy input and label whose gradients match the observed gradient:

$$x'^*, y'^* = \arg \min_{x', y'} \|\nabla W' - \nabla W\|^2 = \arg \min_{x', y'} \left\| \frac{\partial \ell(F(x', W), y')}{\partial W} - \nabla W \right\|^2 \quad (4.1)$$

This rests on the assumption that minimizing gradient distance also reduces the distance to the true input. The dummy input and label are updated iteratively until the gradient difference falls below a specified threshold. Algorithm 1 presents the full algorithm.

Algorithm 1 Deep Leakage from Gradients.

Require: $F(\mathbf{x}; W)$: Differentiable machine learning model; W : parameter weights; ∇W : gradients calculated by training data

Ensure: private training data $\mathbf{x}, \mathbf{y}$

```

1: procedure DLG( $F, W, \nabla W$ )
2:    $\mathbf{x}'_1 \leftarrow \mathcal{N}(0, 1)$ ,  $\mathbf{y}'_1 \leftarrow \mathcal{N}(0, 1)$  ▷ Initialize dummy inputs and labels.
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $\nabla W'_i \leftarrow \partial \ell(F(\mathbf{x}'_i, W_i), \mathbf{y}'_i) / \partial W_i$  ▷ Compute dummy gradients.
5:      $\mathbb{D}_i \leftarrow \|\nabla W'_i - \nabla W\|^2$ 
6:      $\mathbf{x}'_{i+1} \leftarrow \mathbf{x}'_i - \eta \nabla_{\mathbf{x}'} \mathbb{D}_i$ ,  $\mathbf{y}'_{i+1} \leftarrow \mathbf{y}'_i - \eta \nabla_{\mathbf{y}'} \mathbb{D}_i$  ▷ Update data to match gradients.
7:   end for
8:   return  $\mathbf{x}'_{n+1}, \mathbf{y}'_{n+1}$ 
9: end procedure
```

Because $\nabla W'$ is itself produced by a backward pass through the network, each update step $\nabla_{x'} \|\nabla W' - \nabla W\|^2$ differentiates through that gradient computation. The optimization is therefore second-order, or double-backpropagation: one backward pass produces $\nabla W'$ from x' , and a second differentiates the matching loss through it with respect to x' .

For batch sizes $N > 1$, a complication arises: the batch gradient is invariant to sample ordering, so $N!$ permutations of the dummy batch produce the same observed gradient. Optimizing all dummy samples simultaneously therefore becomes slow and unstable. DLG addresses this by updating only a single sample per iteration, which resolves the combinatorial ambiguity and yields stable convergence.

4.2 iDLG

iDLG [3] improves on DLG by recovering the ground-truth label analytically from the observed gradients, removing the need to optimize dummy labels and reducing the reconstruction problem to optimizing the dummy input alone.

The label recovery procedure assumes the model uses cross-entropy loss with a softmax classifier. For an input $\mathbf{x}$ with ground-truth class c and logits $\mathbf{y} = [y_1, y_2, \dots]$, the loss is

$$\ell(\mathbf{x}, c) = -\log \frac{e^{y_c}}{\sum_j e^{y_j}},$$

Let $p_i = \frac{e^{y_i}}{\sum_j e^{y_j}}$ denote the softmax probability of class i . Differentiating with respect to each logit then gives

$$g_i = \frac{\partial \ell(\mathbf{x}, c)}{\partial y_i} = \begin{cases} p_i - 1, & \text{if } i = c \\ p_i, & i \neq c \end{cases}$$

since p_i is between 0 and 1 the gradient for the ground truth class becomes $g_c = p_c - 1 < 0$ while $g_i = p_i > 0$ for all other classes. Therefore, the ground-truth label is uniquely identified by the negative logit gradient.

However, in practice, the shared information consists of gradients with respect to the model parameters rather than the logits themselves. To recover the label from the shared gradients, iDLG relates the logit gradients to the gradients of the final-layer weights.

Consider a network with L layers and output layer $y_i = (\mathbf{W}_L^i)^\top a_{L-1} + b_L^i$, where a_{L-1} denotes the activation vector of the second-to-last layer. Applying the chain rule gives

$$\nabla \mathbf{W}_L^i = \frac{\partial \ell(\mathbf{x}, c)}{\partial \mathbf{W}_L^i} = \frac{\partial \ell(\mathbf{x}, c)}{\partial y_i} \frac{\partial y_i}{\partial \mathbf{W}_L^i} = g_i a_{L-1}.$$

Thus, the gradient of each final-layer weight vector is equal to the corresponding logit gradient scaled by the same activation vector a_{L-1} . When the entries of a_{L-1} are non-negative, as is the case for activation functions such as ReLU and Sigmoid, the sign pattern of $\nabla \mathbf{W}_L^i$ is determined entirely by the sign of g_i .

Since the logit gradient of the ground-truth class is negative while all other logit gradients are positive, the corresponding final-layer weight gradient can be distinguished directly from the remaining classes. Consequently, the ground-truth label can be recovered from the shared gradients without performing any optimization.

This label recovery procedure relies on three assumptions: a batch size of one, cross-entropy loss with a softmax classifier, and non-negative activations in the second-to-last layer. Once the label has been recovered, the reconstruction proceeds exactly as in DLG, except that only the dummy input is optimized while the label remains fixed. Algorithm 2 summarizes the complete procedure.

Algorithm 2 Improved Deep Leakage from Gradients (iDLG)

Require: $F(\mathbf{x}; \mathbf{W})$: Differentiable learning model, $\mathbf{W}$: Model parameters, $\nabla \mathbf{W}$: Gradients produced by private training datum $(\mathbf{x}, c)$, N : maximum number of iterations. η : learning rate.

Ensure: $(\mathbf{x}', c')$: Dummy datum and label.

- 1: $c' \leftarrow i$ **s.t.** $\nabla \mathbf{W}_L^{i^T} \cdot \nabla \mathbf{W}_L^j \leq 0, \forall j \neq i$ ▷ Extract the ground-truth label.
- 2: $\mathbf{x}' \leftarrow \mathcal{N}(0, 1)$ ▷ Initialize the dummy datum.
- 3: **for** $i \leftarrow 1$ **to** N **do**
- 4: $\nabla \mathbf{W}' \leftarrow \partial \ell(F(\mathbf{x}'; \mathbf{W}), c') / \partial \mathbf{W}$ ▷ Calculate the dummy gradients.
- 5: $L_G = \|\nabla \mathbf{W}' - \nabla \mathbf{W}\|_F^2$ ▷ Calculate the loss (difference between gradients).
- 6: $\mathbf{x}' \leftarrow \mathbf{x}' - \eta \nabla_{\mathbf{x}'} L_G$ ▷ Update the dummy datum.
- 7: **end for**

4.3 Networks

The effectiveness of gradient inversion depends not only on the shared gradients themselves but also on the architecture that generates them. Network architectures differ in depth, connectivity, and feature extraction mechanisms, which influence how reconstruction-related information is encoded in the gradients. To evaluate whether masking remains effective across different settings, this thesis considers three representative convolutional architectures: LeNet, ResNet, and VGG.

4.3.1 Convolutional Neural Networks

All architectures considered in this work are convolutional neural networks (CNNs). CNNs are specifically designed to exploit the spatial structure of images by applying convolutional operations, essentially sliding small filters across an image to extract local patterns such as edges, textures, and shapes [10].

Figure 4.1 illustrates a typical CNN pipeline for image classification. The input image is processed through a series of convolutional and pooling layers, extracting hierarchical features (feature maps). The features are flattened and passed through fully connected layers, culminating in a softmax output layer that produces a probability distribution over the target classes (e.g., horse, zebra, dog).

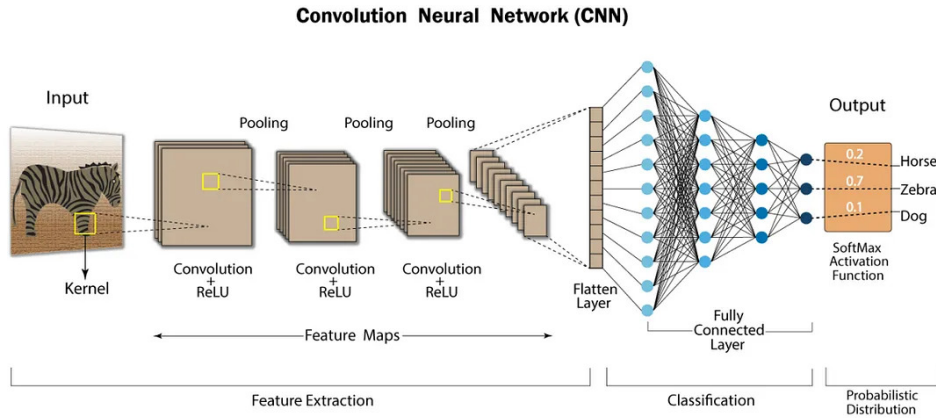


Figure 4.1. CNN classification pipeline. Convolutional and pooling layers extract increasingly abstract feature representations from the input image, which are mapped to class probabilities by the fully connected classifier and softmax output layer. [10]

In the context of gradient inversion, CNNs are of particular interest because the gradients generated during training reflect how visual information is represented within the network. Through successive convolutional layers, low-level patterns such as edges and textures are transformed into increasingly abstract feature representations. As a result, the shared gradients may reveal information about the original training sample, which gradient inversion attacks attempt to reconstruct.

4.3.2 LeNet

LeNet [11] is one of the earliest convolutional neural network architectures and serves as a natural baseline for gradient inversion experiments. It is particularly relevant because several influential gradient inversion studies, including DLG, iDLG, and Inverting Gradients, evaluate their attacks on LeNet-style architectures. Including LeNet therefore allows comparison with prior work while validating the implementation in a simpler setting.

The variant used in this work consists of three convolutional layers, all using 5×5 kernels with same padding ($\lfloor 5/2 \rfloor = 2$). The first two layers use stride 2 for spatial downsampling, while the third uses stride 1. Each convolutional layer produces 12 feature maps and is followed by a sigmoid activation function. The convolutional body is followed by a single fully connected output layer, whose input size is inferred dynamically from the spatial dimensions of the input.

From a reconstruction perspective, LeNet provides a relatively controlled setting. Its shallow depth and sequential structure make the relationship between input pixels and parameter gradients more direct than in deeper modern architectures. At the same time, its sigmoid activations can introduce saturation effects, which may influence optimization during gradient inversion. LeNet therefore serves both as a reference point for prior work and as a baseline for evaluating how reconstruction behaviour changes in deeper architectures.

4.3.3 ResNet

Residual Networks (ResNet) [12] extend conventional CNNs by introducing residual connections that allow information and gradients to bypass intermediate layers. This enables successful optimization of substantially deeper networks than was previously possible, which has led to its widespread adoption in modern deep learning.

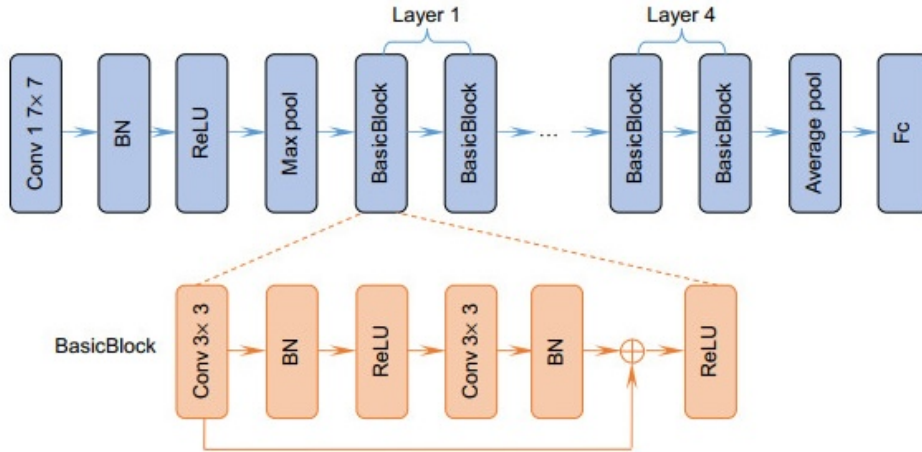


Figure 4.2. Architecture of ResNet-18. The network consists of an initial convolutional stage followed by four groups of residual blocks. Each residual block contains two 3×3 convolutions and an identity skip connection that adds the block input to its output before activation. The architecture concludes with average pooling and a fully connected classifier [13].

The fundamental building block of ResNet is the residual block, which computes

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \quad (4.2)$$

where $\mathbf{x}$ is the block input and $\mathcal{F}$ represents a learnable residual mapping parameterized by weights W_i . The addition of the input term creates an identity pathway that bypasses the intermediate layers, enabling the block to learn residual corrections to the input representation.

ResNet architectures are commonly divided into two families. Shallower variants, such as ResNet-18 and ResNet-34, employ basic blocks consisting of two consecutive 3×3 convolutional layers. Deeper variants, including ResNet-50, ResNet-101, and ResNet-152, use bottleneck blocks that reduce the

channel dimension with a 1×1 convolution, followed by a 3×3 convolution, and then restore the original dimensionality using another 1×1 convolution. This design reduces computational cost while maintaining representational capacity.

The residual formulation also changes how gradients propagate through the network. Differentiating Equation (4.2) with respect to the block input yields

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + I \quad (4.3)$$

where I denotes the identity matrix. The identity term provides a direct path for gradient propagation, which helps reduce the risk of vanishing gradients in deep networks.

From a reconstruction perspective, residual connections change how gradient information is distributed across layers. Since gradients can propagate through multiple paths, input-related information may be spread across the network rather than concentrated in a small number of sequential layers. Masking may therefore affect ResNet differently than plain convolutional networks, because removing one layer group can also remove information that interacts with several residual paths.

4.3.4 VGG

Visual Geometry Group (VGG) networks [14] are deep convolutional networks characterized by a simple and uniform sequential design. Unlike ResNet, VGG does not contain residual connections, meaning gradients propagate through a single sequential chain of layers.

The architecture consists of a sequence of convolutional layers interleaved with max-pooling operations. As the depth increases, the spatial resolution of the feature maps is progressively reduced while the number of channels increases, allowing the network to learn increasingly abstract feature representations. The extracted features are subsequently processed by fully connected layers to produce the final class predictions.

A key design principle of VGG is the repeated use of small 3×3 convolutional filters. Stacking multiple such convolutions increases the effective receptive field while requiring fewer parameters than a single large convolution. The additional layers also introduce extra nonlinearities, increasing the representational capacity of the network. Several VGG variants exist, including VGG-11, VGG-13, VGG-16, and VGG-19, differing primarily in the number of convolutional layers. Despite these differences, all variants follow the same sequential architecture.

From the perspective of gradient inversion, VGG exhibits a purely sequential flow of information and gradients, in contrast to the multiple interacting pathways present in ResNet architectures. Because both the forward and backward passes follow a single path through the network, the relationship between individual layers and the resulting gradients is more straightforward to analyze. Consequently, the effect of removing gradient information from particular layers can be studied more directly than in architectures with residual pathways.

4.4 Pretrained Networks

In practical federated learning settings, models are often initialized using pretrained weights rather than trained entirely from random initialization. Pretraining refers to first optimizing a network on a large external dataset and subsequently adapting it to a target task. This approach has become standard practice in deep learning because the learned representations often transfer effectively across related tasks.

From the perspective of gradient inversion, pretraining changes the relationship between the input and the resulting gradients. In a randomly initialized network, gradients are computed with respect to parameters that have not yet learned meaningful structure from data. In contrast, a pretrained network contains convolutional filters that have already adapted to visual patterns present in the pretraining dataset.

Studies of learned representations have shown that early convolutional layers tend to capture low-level features such as edges, textures, and simple shapes, while deeper layers encode increasingly abstract and semantic information [15]. Consequently, pretraining changes the feature representations through which gradients are computed, which may alter the information contained in the observed gradients relative to a randomly initialized network.

This change does not necessarily make reconstruction easier or more difficult. On one hand, learned representations may preserve information that is strongly correlated with the input. On the other hand, the resulting gradients are influenced by increasingly abstract features that are not directly aligned with pixel-level reconstruction. The overall effect of pretraining on gradient inversion therefore depends on how these competing factors influence the information contained in the observed gradients.

Pretraining may also influence the magnitude and distribution of gradients throughout the network. Since pretrained models already contain learned representations, the gradients produced during training can differ substantially from those of randomly initialized models. In particular, information relevant to reconstruction may become concentrated in a smaller subset of gradient directions or layers. Consequently, pretraining can affect not only the amount of information contained in the gradients, but also how that information is distributed across the network.

Comparing randomly initialized and pretrained networks therefore provides insight into how learned representations influence the information available for reconstruction and whether gradient inversion remains effective in settings that more closely resemble realistic federated learning deployments.

4.5 Masking

As previously stated gradient inversion attacks are formulated as optimization problems that attempt to match the observed gradients of a target image. In high-dimensional networks, however, the observed gradients may differ substantially in magnitude and numerical sensitivity, and not all gradient entries necessarily contribute equally to reconstruction. Some gradients may provide useful information about the input, while others may introduce poorly conditioned optimization directions that make reconstruction more difficult.

Masking addresses this issue by selectively removing subsets of the observed gradients before reconstruction. In addition to reducing the number of gradients that must be matched during optimization, masking may improve numerical conditioning by suppressing gradient entries that contribute little useful reconstruction information.

However, masking introduces a fundamental tradeoff. Removing gradients may improve optimization stability and reduce computational cost, but it also reduces the information available for reconstruction. An effective masking strategy must therefore remove gradients that are redundant or poorly conditioned while preserving those that contribute independent information about the input.

This defines the central challenge of masking: improving optimization stability while preserving enough information for reconstruction to remain possible.

4.5.1 Necessary Conditions for Reconstruction

Previous work has derived lower bounds for reconstruction by comparing the number of available equations with the number of unknown input variables [5]. However, such bounds do not account for dependencies between equations.

This distinction becomes especially important when gradients are masked. Even if the number of retained gradients exceeds the number of unknown input variables, reconstruction may still be impossible if many of the retained gradients provide redundant information. Consequently, reconstruction depends not only on the number of observed gradients, but also on how many independent constraints they provide.

Let the unknown input image be represented as $x \in \mathbb{R}^d$ where the number of unknowns is determined by the number of channels C , height H , and width W : $d = C \times H \times W$.

For LFW and CIFAR-100, this gives $d = 3 \times 32 \times 32 = 3072$. After masking, let the observed gradient vector be represented as $g(x) \in \mathbb{R}^n$, where n denotes the number of retained gradient entries.

A necessary condition for reconstruction is that the number of observed gradient equations must be at least as large as the number of unknown input variables, that is, $n \geq d$. However, this condition alone is not sufficient. Many gradient equations may be linearly dependent and therefore provide redundant information. This means that even if the number of observed gradients exceeds the number of unknowns, reconstruction may still fail.

To quantify the number of independent constraints retained after masking, the reconstruction problem is analyzed through the rank of the Jacobian of the observed gradients with respect to the input. The Jacobian rank measures the number of independent input directions constrained by the observed gradients and therefore provides a more informative criterion for reconstruction than equation counts alone.

This follows directly from a fundamental result in linear algebra:

Theorem 1 (Linear independence and rank). *Let $\mathbf{v}_1, \dots, \mathbf{v}_n \in \mathbb{F}^m$ be given, and let $\mathbf{A} \in \mathbb{F}^{m \times n}$ be the matrix with columns $\mathbf{v}_1, \dots, \mathbf{v}_n$. The sequence of vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ is linearly independent if and only if the matrix $\mathbf{A}$ has rank n . [16, Theorem 7.8]*

In the context of gradient inversion, this means that reconstruction depends on the number of independent constraints provided by the observed gradients. This can be formalized by constructing the Jacobian matrix of the observed gradients with respect to the input: $J(x) = \partial g(x) / \partial x \in \mathbb{R}^{n \times d}$.

Since the input contains d unknown variables, the Jacobian can have rank at most d . Reconstruction of a specific input x is therefore only possible if the Jacobian evaluated at that image has full column rank, $\text{rank}(J(x)) = d$, meaning each input dimension contributes an independent degree of freedom. Full column rank at one input does not imply the same holds for all inputs.

If instead $\text{rank}(J(x)) < d$, then there exist multiple different inputs that produce the same observed gradients, making the reconstruction problem underdetermined.

This relationship can be understood through the rank-nullity theorem:

Theorem 2 (Rank-nullity theorem). *Let $\mathbb{F}$ be a field and $A \in \mathbb{F}^{m \times n}$ be a matrix. Then*

$$\rho(\mathbf{A}) + \text{null}(\mathbf{A}) = n,$$

where $\rho(\mathbf{A})$ denotes the rank of the matrix $\mathbf{A}$ and $\text{null}(\mathbf{A})$ its nullity. [16, Theorem 10.10]

This theorem shows that any reduction in rank directly increases the null space of the system. In the reconstruction setting, this means there are more possible input images that satisfy the same observed gradients, and the solution is no longer unique.

This makes masking particularly delicate: removing gradients may improve optimization stability, but any reduction in rank makes the reconstruction problem underdetermined. In practice, this means that an effective masking strategy should aim to remove redundant or noisy gradients while preserving those that contribute independent constraints. Section 5.1 verifies the full-rank condition empirically for all configurations used in the experiments.

Full column rank therefore identifies the point below which reconstruction is theoretically impossible, but it does not identify the point above which reconstruction becomes reliable in practice. The experiments in this thesis examine this gap directly by measuring reconstruction quality as a function of the number of retained gradient entries, including in the regime where the full-rank condition is already satisfied.

In practice, the Jacobian entries are floating-point approximations computed numerically in PyTorch. Exact rank computation requires exact arithmetic: rounding errors mean that entries which should

be exactly zero may appear as small nonzero values, making it impossible to determine the true rank reliably.

Instead, the rank is approximated numerically using singular value decomposition (SVD), where the numerical rank is defined as the number of singular values above a chosen threshold. Intuitively, singular values quantify how strongly the matrix spans independent directions in the input space. Large singular values correspond to informative directions, while singular values close to zero indicate redundancy. This provides a practical way to estimate how much reconstructive information remains after masking.

4.5.2 Layer Importance in Gradient Inversion

The rank-based perspective helps explain why different layers may contribute differently to reconstruction. Gradients from different parts of a network are not equally informative about the input image. Some layers are closely tied to low-level image structure, while others encode higher-level representations that are useful for classification but less directly tied to the input pixels. As a result, masking a layer can either remove useful reconstruction constraints or improve optimization by suppressing gradients that are poorly conditioned or weakly informative.

The first convolutional layer is often important because it operates directly on the input image. Its gradients are therefore closely connected to local pixel-level structure such as edges, textures, and colour patterns. In architectures where this layer provides a stable and informative mapping from input pixels to gradients, removing it can significantly reduce reconstruction quality. This is especially relevant for residual networks, where skip connections allow early-layer gradient information to remain accessible throughout the network.

However, the importance of the first convolutional layer is not universal. A layer can contain information and still make optimization harder if its gradient entries are poorly scaled, noisy, or badly conditioned relative to the rest of the gradient vector. In plain sequential architectures such as VGG, masking early layers may therefore improve reconstruction if the optimizer benefits more from removing unstable constraints than it loses from discarding their information. Layer importance must consequently be evaluated empirically rather than assumed from position alone.

The two previous paragraphs seemingly present a contradiction: early-layer gradients depend directly on the input and should therefore be highly informative, yet removing them sometimes improves reconstruction. This can be understood by considering that although the first-layer gradient contains the input itself through the forward-pass activations, it is also determined by the backpropagated error signal arriving at that layer. As with all network gradients, the resulting signal therefore depends on both forward and backward propagation. In a deep sequential network with randomly initialized weights, this backpropagated signal may be poorly conditioned, becoming very small, highly variable, or poorly scaled relative to gradients from later layers. Consequently, although early-layer gradients contain direct information about the input, the quality of that information is influenced by the error signal through which it is transmitted. Masking such gradients may therefore improve reconstruction by removing poorly conditioned optimization constraints while retaining more stable gradient information from deeper layers.

This also explains why the benefit from masking early layers is smaller for pretrained networks. When subsequent layers carry structured, learned weights, the backpropagated signal reaching early layers is more stable, making early-layer gradients more informative. Since early-layer gradients are now better conditioned and more informative, masking them removes useful gradient information rather than noise.

The final fully connected layer plays a distinct role in gradient inversion. Unlike the convolutional layers, which primarily contain information about the input image, the classifier layer contains information about the target label. For iDLG-style attacks, its gradients enable analytical label recovery directly from the observed gradients, making the classifier layer important for reconstruction despite its limited connection to the image content itself [3].

Overall, masking decisions should be understood as architecture- and optimizer-dependent. Preserving a layer is beneficial only when the information it contributes improves reconstruction more than it worsens the conditioning of the optimization problem. The experiments in this thesis therefore evaluate layer masking across both residual and plain convolutional networks to determine where this balance lies.

4.6 Non-Convex Optimization

Gradient inversion reduces to finding the dummy input x' that minimizes the gradient matching loss $\mathcal{L}(x')$. How hard that search is depends on the geometry of the objective: for a convex loss, any local minimum is also a global minimum, so the optimizer finds the same solution regardless of where it starts. Gradient matching does not have this property, because the loss is non-convex. For a convex function, the value at any convex combination of two points lies at or below the straight line connecting them; Figure 4.3 contrasts this bowl-shaped geometry with the non-convex landscape arising in gradient inversion, where poor local minima, saddle points, and lower-loss solutions can coexist. Different starting points can therefore lead the optimizer to entirely different outcomes.

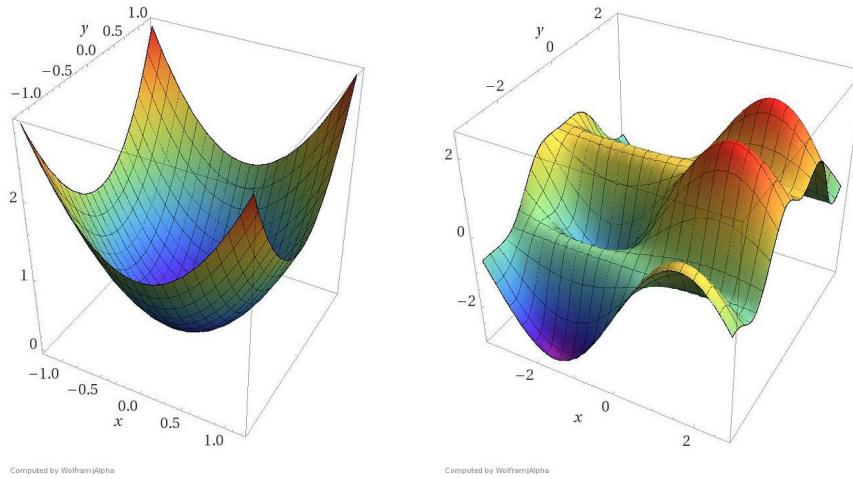


Figure 4.3. Comparison of convex and non-convex optimization landscapes. Convex objectives (left) have a single global minimum, while non-convex objectives (right) may contain multiple local minima and saddle points. The gradient matching objective used in gradient inversion is generally non-convex [17].

The gradient matching loss is non-convex because computing $\nabla \mathbf{W}(x')$ requires backpropagating through non-linear activation functions. For the L2 objective, if the network contained no non-linearities, $\nabla \mathbf{W}(x')$ would vary linearly with x' and the squared Euclidean distance to a fixed target gradient would be a convex quadratic in x' . Non-linearity breaks this structure [4], producing a loss surface with many local minima and saddle points. Gradient descent may converge to a poor local minimum, or stall near a saddle point.

Saddle points compound the difficulty: at such points the gradient vanishes although the point is not a minimum, meaning optimizers receive little directional signal and can stall for many iterations.

The combined effect is a search problem with no convergence guarantees: the optimizer may settle in a poor local minimum or stall near a saddle point far from the global optimum. Even when optimization succeeds, the gradient matching objective does not measure image similarity directly, meaning that a low gradient matching loss does not necessarily imply an accurate reconstruction of the ground truth image. This makes the choice of gradient matching loss important, since it determines which differences between dummy and observed gradients are emphasized during optimization.

4.7 Gradient Matching Loss Functions

Since the optimization is driven entirely by the gradient matching objective, the choice of objective determines how agreement between the dummy and observed gradients is quantified. This agreement should not be interpreted as a direct measure of image similarity, since the objective compares gradients rather than pixels, and a low gradient matching loss therefore does not necessarily imply an accurate reconstruction of the ground truth image.

This thesis considers two gradient matching loss functions: L2 loss, which measures the squared Euclidean distance between gradient vectors, and cosine-similarity loss, which measures their directional alignment. Both losses can be computed over the full gradient vector or restricted to a selected subset of entries when masking is applied.

Let $\nabla \mathbf{W}$ denote the gradient observed from the true data, and let $\nabla \mathbf{W}'$ denote the gradient induced by the dummy input. These two gradients are the quantities compared by the matching objective. When masking is applied, the comparison is restricted to the retained gradient entries.

4.7.1 L2 Gradient Matching Loss

The L2 loss measures the squared Euclidean distance between the dummy gradient and the observed gradient:

$$\mathcal{L}_{L2}(\nabla \mathbf{W}', \nabla \mathbf{W}) = \sum_i (\nabla \mathbf{W}'_i - \nabla \mathbf{W}_i)^2, \quad (4.4)$$

where i indexes the selected gradient entries. The objective penalizes discrepancies in both the magnitude and direction of the gradient vectors. Consequently, minimizing the loss encourages the dummy gradient to match the observed gradient in both respects. DLG and iDLG both use this loss.

Because the objective depends on absolute gradient values, entries with large magnitudes exert disproportionate influence on the optimization. In deeper networks, where gradient norms can differ substantially across layers, optimization can become overly dependent on a small number of dominant gradient components, making the reconstruction process more sensitive to scale differences between layers.

4.7.2 Cosine Similarity Gradient Matching Loss

Cosine-similarity measures the directional alignment between two gradient vectors, independently of their magnitudes. For flattened gradient vectors $\mathbf{g}'$ and $\mathbf{g}$, it is defined as:

$$\cos(\mathbf{g}', \mathbf{g}) = \frac{\mathbf{g}' \cdot \mathbf{g}}{\|\mathbf{g}'\|_2 \|\mathbf{g}\|_2}. \quad (4.5)$$

Since cosine-similarity is maximized at 1 when the vectors are perfectly aligned, it is converted to a loss as:

$$\mathcal{L}_{cos}(\mathbf{g}', \mathbf{g}) = 1 - \frac{\mathbf{g}' \cdot \mathbf{g}}{\|\mathbf{g}'\|_2 \cdot \|\mathbf{g}\|_2 + \epsilon}, \quad (4.6)$$

where ϵ is a small constant greater than 0 included for numerical stability.

Because both vectors are normalized before comparison, cosine-similarity is scale-invariant: rescaling either $\mathbf{g}'$ or $\mathbf{g}$ leaves the loss unchanged. This reduces the influence of gradient magnitude differences across layers and makes the objective depend primarily on directional agreement.

4.7.3 Comparison

The difference between the two objectives is most apparent in the masked setting. Restricting the objective to a subset of gradient entries reduces the number of terms contributing to the L2 objective, which can increase the influence of the remaining large-magnitude entries. Cosine-similarity is less sensitive to such magnitude differences because both vectors are normalized before comparison and the objective therefore depends primarily on directional agreement. Whether this scale-invariance is beneficial in practice depends on whether the most informative gradient entries are also the largest, which is an empirical question evaluated in the experiments.

Although the full-rank condition of Section 4.5.1 confirms that the gradients impose sufficient independent constraints on the input, the relationship between gradient mismatch and image error is not straightforward. The mapping from a gradient residual back to pixel-space reconstruction error is generally poorly conditioned, meaning that a small but non-zero gradient matching loss can still correspond to a reconstruction that differs substantially from the true input. Consequently, the loss value alone is an unreliable indicator of reconstruction quality, which is instead assessed using the image-based evaluation metrics introduced in Section 4.10.

4.8 Optimizers

The gradient matching losses introduced in Section 4.7 define the reconstruction objective. However, the objective itself does not specify how the dummy image is updated during optimization. Gradient inversion therefore requires an optimizer to solve:

$$x'^* = \arg \min_{x'} \mathcal{L}_{\text{grad}}(\nabla \mathbf{W}', \nabla \mathbf{W}), \quad (4.7)$$

where $\mathcal{L}_{\text{grad}}$ denotes the gradient matching loss from Section 4.7. The optimization variables are the pixels of the dummy image x' , while the network parameters remain fixed.

As discussed in Section 4.6, the reconstruction objective is highly non-convex and may contain poor local minima, saddle points, and regions where the gradient provides little useful information. Consequently, different optimization strategies can converge to substantially different reconstructions, even when minimizing the same objective. Importantly, the objective is also fully deterministic: the target gradient $\nabla \mathbf{W}$ is fixed throughout the attack, so evaluating the loss at the same dummy image always returns the same value.

This thesis evaluates two optimizers: L-BFGS, used in the original iDLG setup, and Signed AdamW, introduced for gradient inversion by Geiping et al. [4]. The two represent different optimization strategies, quasi-Newton versus adaptive first-order, applied to the same reconstruction objective under otherwise identical conditions. The following subsections describe these methods and discuss their application to gradient inversion.

4.8.1 L-BFGS

The L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno) optimizer [18] is motivated by the limitation of first-order methods: they determine updates solely from the current gradient, without accounting for how the gradient changes across the loss surface. This curvature information is captured by the Hessian $\nabla^2 \mathcal{L}(\mathbf{x})$, which contains the second partial derivatives of the loss. Incorporating it allows an optimizer to adapt each step to the local geometry rather than relying on gradient direction alone, which can substantially reduce the number of iterations needed on smooth objectives.

However, directly storing and inverting the Hessian is generally impractical because it contains d^2 entries, where d denotes the number of optimization variables. L-BFGS addresses this by maintaining an implicit approximation of the inverse Hessian using the m most recent parameter and gradient differences, which capture how the local curvature has changed across recent iterations. Approximating the inverse Hessian from gradient differences, rather than computing it directly, is what places L-BFGS in the class of quasi-Newton methods; using only the m most recent differences is what keeps it memory-efficient.

At iteration t , the update takes the form

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta H_t \nabla \mathcal{L}(\mathbf{x}_t), \quad (4.8)$$

where H_t is a positive-definite approximation to the inverse Hessian $[\nabla^2 \mathcal{L}(\mathbf{x}_t)]^{-1}$ and η is the step size. Positive-definiteness ensures that $-H_t \nabla \mathcal{L}$ is a descent direction, while the step size or line search determines whether the update decreases the loss in practice. The history length m trades curvature approximation against memory usage. In this thesis, the default PyTorch value of $m = 100$ is used.

L-BFGS is particularly well suited to gradient inversion because the reconstruction objective is deterministic. The target gradient is computed only once, while the network parameters remain fixed throughout the attack. As a result, evaluating the objective at the same dummy image always produces the same loss value and gradient. This consistency allows L-BFGS to construct a meaningful curvature approximation from successive optimization steps, allowing faster convergence on smooth, well-conditioned regions of the loss surface where the curvature approximation is reliable.

4.8.2 Adam-based Optimizers

The first-order optimizer used in this thesis is Signed AdamW, a variant of Adam designed for gradient inversion. To motivate this optimizer, it is useful to first introduce Adam and AdamW.

Adam

The Adam (Adaptive Moment Estimation) optimizer, proposed by Kingma and Ba [19], is a first-order optimizer originally developed for stochastic training objectives that combines momentum and adaptive learning rates. Momentum smooths the optimization path by maintaining a running average of past gradients, while adaptive learning rates scale each update according to the historical magnitude of the gradients.

Let $\mathbf{g}_t = \nabla \mathcal{L}(\mathbf{x}_t)$ denote the gradient of the loss function at iteration t . Adam computes an exponential moving average of the gradients: $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$ known as the first moment estimate, and an exponential moving average of the squared gradients: $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$, known as the second moment estimate. After bias correction the final parameter update takes the form

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \quad (4.9)$$

where η is the learning rate and ϵ is a small constant added for numerical stability.

AdamW

AdamW [20] modifies Adam by decoupling weight decay from the adaptive gradient update. As a result the update becomes:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} - \eta \lambda \mathbf{x}_t \quad (4.10)$$

where λ is the weight decay coefficient, so that weight decay acts independently of Adam's adaptive learning rate.

Signed AdamW

For gradient inversion, Geiping et al. [4] replace the gradient with its element-wise sign before applying the AdamW update:

$$\mathbf{g}_t \leftarrow \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}). \quad (4.11)$$

Instead of using the magnitude of each gradient component, the optimizer uses only its direction. Large and small gradient components are treated equally, preventing a few high-magnitude entries from dominating the optimization process.

This attribute is particularly relevant for gradient inversion, where the gradient matching objective is often ill-conditioned and may contain large variations in gradient scale across optimization variables. By discarding magnitude information, Signed AdamW emphasizes directions that remain consistent across iterations rather than directions associated with large gradient values.

The sign-transformed gradients are then passed through Adam’s momentum and adaptive averaging mechanisms in the usual way. Although the sign operation reduces the influence of AdamW’s adaptive scaling, the momentum and smoothing behavior of the optimizer are retained. Geiping et al. report that this modification, when combined with cosine-similarity loss, substantially improves optimization stability and reconstruction quality in gradient inversion attacks. Signed AdamW is therefore used as the first-order optimizer in this thesis.

4.9 Activation functions

Activation functions introduce non-linearity into neural networks. Without them, each layer would perform only a linear transformation of its input, and even a deep network composed of multiple such layers would remain equivalent to a single linear mapping. This would severely limit the complexity of the patterns the network could represent and learn.

By applying a non-linear activation function after each layer, the network becomes capable of modeling more complex relationships between inputs and outputs. Activation functions therefore play a fundamental role in deep learning, as they enable neural networks to learn hierarchical and non-linear representations of data.

In addition to increasing representational power, activation functions also influence optimization behavior. Different activation functions affect gradient propagation, convergence speed, and numerical stability, and therefore have an important impact on both training dynamics and model performance.

In this thesis, two activation functions are of particular relevance: the sigmoid function, used in the LeNet network, and the Rectified Linear Unit (ReLU), used in the other networks and in most modern convolutional neural networks.

4.9.1 Sigmoid

The sigmoid activation function maps any real-valued input to the interval $]0, 1[$ and is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (4.12)$$

Because of its bounded output range, the sigmoid function is often interpreted as producing a probability-like value.

Despite this, sigmoid has several disadvantages in deep networks. For large positive or negative inputs, the function saturates, meaning that its gradient becomes very small. This can lead to the vanishing gradient problem, where gradients diminish as they are propagated backward through the network, making optimization slower and less stable. In addition, sigmoid outputs are not zero-centered, since they are always positive. As a result, the gradients in subsequent layers can become more correlated and less balanced, which may reduce the efficiency of gradient-based optimization.

In this thesis, only the custom LeNet network used in both the DLG and iDLG papers uses the sigmoid activation function.

4.9.2 ReLU

The Rectified Linear Unit (ReLU) is defined as

$$\text{ReLU}(x) = \max(0, x). \quad (4.13)$$

It maps negative inputs to zero while passing positive inputs through unchanged. Due to its simplicity and strong performance, ReLU has become one of the most widely used activation functions in modern deep learning.

Compared to sigmoid, ReLU offers several advantages. Most notably, it does not saturate for positive inputs, which helps mitigate the vanishing gradient problem and allows gradients to propagate more

effectively during training [21], often resulting in faster convergence and improved optimization behavior in deep networks. ReLU is also computationally efficient, as it only requires a simple thresholding operation.

A limitation of ReLU is that neurons can become permanently inactive if their inputs remain negative, causing their gradients to be zero. This phenomenon is commonly referred to as the dying ReLU problem. Nevertheless, ReLU is generally preferred over sigmoid in modern neural networks, as it tends to provide more stable and efficient optimization in practice.

In this thesis, ReLU is particularly relevant as it is the activation function of choice for all ResNet and VGG models, which form the backbone of the gradient inversion experiments in this thesis.

4.10 Evaluation metrics

Evaluating the effectiveness of these attacks requires quantifying how closely reconstructed images resemble the original training data. While visual inspection can provide qualitative insights, objective evaluation requires numerical metrics that capture differences in pixel values and perceptual similarity. This work considers commonly used image reconstruction metrics, focusing on Peak Signal to Noise Ratio (PSNR), which measures pixel-level similarity between reconstructed and ground-truth images.

4.10.1 MSE

Mean Squared Error (MSE) [22] measures the average squared pixel-wise difference between a reconstructed image $\hat{x}$ and the ground-truth image x :

$$\text{MSE} = \frac{1}{d} \sum_{i=1}^d (\hat{x}_i - x_i)^2, \quad (4.14)$$

where $d = C \times H \times W$ is the total number of pixel values across all channels. Since images are normalized to $[0, 1]$, the maximum possible per-pixel difference is 1, so MSE lies in $[0, 1]$, with 0 indicating a perfect reconstruction.

MSE penalizes deviations quadratically, making it sensitive to outlier pixels. It treats every pixel position equally, regardless of its perceptual importance, and is affected by global brightness or contrast shifts even when the underlying structure is well recovered [22]. Despite these limitations, MSE is widely used due to its simplicity and its direct relationship to PSNR, which is defined in terms of MSE in the following subsection.

4.10.2 PSNR

Peak Signal-to-Noise Ratio (PSNR) is a widely used metric for measuring the similarity between a reconstructed image and its corresponding ground-truth image. It is based on the MSE and defined as:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right), \quad (4.15)$$

where $\text{MAX} = 1$ since images are normalized to $[0, 1]$.

Higher PSNR values indicate greater similarity between images, as a lower reconstruction error results in a higher ratio. However, as a purely pixel-level measure it does not necessarily reflect perceptual quality, a limitation addressed by SSIM.

The logarithmic transformation also has distributional consequences relevant to parametric testing; these are explained in Section 5.4.

4.10.3 SSIM

Structural Similarity Index Measure (SSIM) [23] is designed to capture how similar two images are in terms of perceived structure, rather than just pixel-wise differences.

Instead of comparing pixels directly, SSIM evaluates local patterns of intensity and decomposes similarity into three components: luminance, contrast, and structure. Applied over local sliding windows; a small patch moved across the image so that SSIM is computed for every pixel neighborhood and then averaged. It is defined per window as:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (4.16)$$

where μ_x and μ_y are the mean intensities, σ_x^2 and σ_y^2 are the variances, and σ_{xy} is the covariance between the two images, all computed over the local window. The constants $C_1 = (K_1L)^2$ and $C_2 = (K_2L)^2$ are included to stabilize the ratio when the denominator terms are close to zero, where L is the dynamic range of the pixel values and $K_1, K_2 \ll 1$ are small constants [23].

The SSIM index lies in $[-1, 1]$, attaining 1 only for identical images; for natural-image reconstructions it is typically positive, where higher values indicate greater similarity. A negative value does not imply less structural relationship than zero, but rather an inverse relationship: it arises when the local covariance term σ_{xy} is negative, so the two windows are anti-correlated and their structural features are inverted relative to one another.

Because PSNR and SSIM capture distinct aspects of image fidelity, specifically absolute pixel error and perceived structural integrity, reporting both metrics provides a comprehensive evaluation of reconstruction quality.

4.10.4 Shapiro-Wilk Normality Test

The Shapiro-Wilk test evaluates the null hypothesis, H_0 , that a sample $x_1, \dots, x_n$ is drawn from a normal distribution [24]. The test statistic is

$$W = \frac{\left(\sum_{i=1}^n a_i x_{(i)} \right)^2}{n \sum_{i=1}^n (x_i - \bar{x})^2}, \quad (4.17)$$

where $x_{(1)} \leq \dots \leq x_{(n)}$ are the order statistics, $\bar{x}$ is the sample mean, and a_i are coefficients derived from the expected values and covariance matrix of the order statistics of a standard normal distribution. $W \in]0, 1]$, with values near 1 indicating agreement with normality and small values indicating departure. Small values of W yield small p -values, providing evidence against the null. A p -value above the significance level α , indicates insufficient evidence to reject normality. At small sample sizes the test has limited power against mild deviations, so a non-rejected null should be interpreted as ruling out gross non-normality rather than establishing normality.

This chapter describes the experimental setup and statistical methodology underlying the reconstruction experiments. The central design is a within-image paired comparison: for each trial, a baseline gradient inversion and a masked variant are run on the same input under otherwise identical conditions, so that differences in reconstruction quality are directly attributable to masking. Before any reconstruction is attempted, the necessary conditions for unique inversion established in Section 4.5.1 are verified for each architecture and dataset.

5.1 Verifying necessary conditions for image reconstruction

Gradient inversion can uniquely recover an input only if the observed gradients impose sufficient independent constraints on all image unknowns. As formalized in Section 4.5.1, this corresponds to a full column rank condition on the Jacobian $J(x) = \frac{\partial g(x)}{\partial x} \in \mathbb{R}^{n \times d}$, where $d = C \times H \times W$ is the number of image unknowns and $g(x)$ is the vector of observed gradient entries. Verifying that this condition still holds after masking ensures that observed PSNR differences between masked and baseline configurations reflect changes in optimization conditioning rather than changes in recoverable information.

However, constructing the full Jacobian is computationally infeasible for the architectures considered in this study. Even in ResNet-18, the smallest evaluated architecture aside from the LeNet baseline, a CIFAR-100 input results in approximately 11 million gradient entries and 3,072 image unknowns, yielding a Jacobian with roughly 34.5 billion entries. In practice, the rank verification is therefore reduced to a smaller problem: finding the minimum number of gradient entries needed to obtain full column rank. The rank is estimated from a growing subset of k selected gradient entries. If a k -row submatrix of $J(x)$ already achieves full column rank d , then $J(x)$ itself must also have full column rank, since adding rows cannot reduce the rank. The minimum value of k at which this first occurs is recorded for each architecture and for each method of selecting gradient entries.

The gradient entries are drawn from a pool comprising all network parameters, including the final fully connected (FC) classification layer. The k entries included in each Jacobian row subset are chosen according to one of two selection methods: `topk_abs`, which selects entries globally by absolute gradient magnitude, and `layer_spread`, which distributes the k entries evenly across layer groups. Because FC gradients tend to be large in magnitude, `topk_abs` can assign disproportionately many of the k entries to FC parameters, whereas `layer_spread` bounds each layer group to an equal share regardless of magnitude. Comparing the two methods therefore reveals how the distribution of selected entries across layers affects the minimum entry count needed to achieve full rank.

Given the selected entries, the Jacobian rows are computed using PyTorch’s `autograd.grad`, differentiating each selected gradient entry with respect to the input x . Numerical rank of the resulting submatrix is then estimated using `torch.linalg.matrix_rank`, which applies the threshold:

$$\tau = \max(\text{atol}, \sigma_{\max} \cdot \text{rtol}), \quad \text{rtol} = \max(m, n) \cdot \varepsilon, \quad (5.1)$$

discarding singular values below τ , where `atol` = 0 by default, $\sigma_{\max}$ is the largest singular value, $\max(m, n)$ is the larger matrix dimension, and ε is the machine precision of the data type (dtype) used in PyTorch.

The threshold in Equation (5.1) depends on the machine precision ε , making the choice of dtype important. Using the default `float32` precision ($\varepsilon \approx 1.19 \times 10^{-7}$) proved insufficient for the rank sweep. As additional Jacobian rows are appended, rounding errors accumulate and the relative tolerance term scales with $\max(m, n)$, increasing the threshold τ . Consequently, singular values corresponding to genuine independent directions may fall below the threshold and become indistinguishable from numerical noise. This can lead to an estimated rank that decreases as more rows are added, despite the fact that matrix rank can only increase or remain unchanged when rows are appended.

To mitigate this issue, all rank computations were performed using `float64` precision ($\varepsilon \approx 2.22 \times 10^{-16}$). The substantially smaller machine epsilon lowers the numerical noise floor and reduces the threshold τ , allowing many more Jacobian rows to be incorporated before signal and noise become indistinguishable. Although `float64` does not eliminate numerical limitations entirely, it extends the range over which the estimated rank exhibits the expected monotonic behavior.

5.2 Gradient Inversion Attacks Implementation

Two experimental reconstruction setups are evaluated: one following iDLG [3] and one following Inverting Gradients [4]. The two configurations differ in their choice of gradient matching objective, optimizer, and iteration budget. Within each configuration, a baseline run (no masking) and a masked run are compared under identical hyperparameter settings. All experiments use batch size 1, since iDLG’s label inference relies on the sign of the final fully connected layer gradient (Section 4.2). Masking strategies are described in Section 5.2.11.

5.2.1 Normalization and Clamping

For VGG and ResNet architectures, inputs are normalized channel-wise. Raw pixel values $x_c \in [0, 1]$ are transformed to $\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c}$, $c = 1, \dots, C$, using per-channel means $\mu \in \mathbb{R}^C$ and standard deviations $\sigma \in \mathbb{R}^C$ computed from the full training set. The CIFAR-100 values follow the standard inverting gradients constants reported in Chapter 3; the LFW constants, for which no standard values exist at the 32×32 resolution used here, were computed directly from the dataset as described in Section 5.2.2. When a pretrained network is used, ImageNet statistics ($\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$) are used in place of the dataset statistics, since the pretrained weights were learned under this normalization.

The dummy image is kept in normalized space throughout optimization. At each iteration, the normalized dummy $\hat{x}'$ is passed directly to the network without any additional transformation. Reconstruction quality metrics are computed on unnormalized images: at each iteration, $\hat{x}'$ is converted back to pixel space via $x' = \hat{x}' \cdot \sigma + \mu$, clamped to $[0, 1]$, and compared against the ground-truth image to compute MSE and PSNR.

Since valid pixel values satisfy $x_c \in [0, 1]$, the corresponding feasible region in normalized space is the per-channel interval $\hat{x}_c \in \left[-\frac{\mu_c}{\sigma_c}, \frac{1-\mu_c}{\sigma_c}\right]$. After each optimizer step, the dummy image is projected onto this region by element-wise clamping,

$$\hat{x}'_c \leftarrow \text{clamp}\left(\hat{x}'_c, -\frac{\mu_c}{\sigma_c}, \frac{1-\mu_c}{\sigma_c}\right), \quad (5.2)$$

keeping the reconstruction within the valid image domain throughout. For VGG and ResNet experiments, all configurations use the same normalization constants and clamping; differences in reconstruction quality between the baseline and masked runs are therefore attributable to the gradient masking alone. LeNet is an exception. Sigmoid activations are designed for raw $[0, 1]$ inputs; applying dataset normalization shifts pixel values into approximately $[-2, +2]$, saturating the sigmoid and zeroing out convolutional gradients, making inversion impossible. LeNet experiments therefore use no normalization ($\mu = 0$, $\sigma = 1$) and skips clamping. Clamping after L-BFGS steps corrupts the Hessian approximation that L-BFGS maintains, causing runs to diverge; pixels instead converge naturally to $[0, 1]$ as the loss drives the reconstruction toward the ground truth.

5.2.2 LFW Normalization Statistics

Unlike CIFAR-100, no standard normalization constants exist for LFW at the 32×32 resolution used here, so they were computed directly from the dataset. The per-channel mean and standard deviation were aggregated over all $13,233 \times 32 \times 32$ pixels per channel, with pixel values in $[0, 1]$ after conversion to a tensor, using population statistics:

$$\mu_c = \frac{1}{N} \sum_{i=1}^N x_{i,c}, \quad \sigma_c = \sqrt{\frac{1}{N} \sum_{i=1}^N x_{i,c}^2 - \mu_c^2},$$

where N is the total number of pixels in channel c across the entire dataset. The resulting values are $\mu = (0.420, 0.365, 0.325)$ and $\sigma = (0.284, 0.258, 0.251)$ for the R, G, and B channels respectively. Each image is then normalized as $\hat{x}_c = (x_c - \mu_c)/\sigma_c$ before being passed to the network.

Together, CIFAR-100 and LFW provide complementary evaluation contexts: one with balanced class structure and diverse natural object categories, the other with severe class imbalance and sensitive biometric content. Both are evaluated at 32×32 resolution throughout the reconstruction experiments.

5.2.3 Initialization

The dummy image is initialized by drawing each entry independently from a standard normal distribution, $\mathcal{N}(0, 1)$. For VGG and ResNet experiments, the dummy is initialized and optimized in normalized space and clamped to the feasible normalized range after each optimizer step. For LeNet, no normalization is applied; the dummy is initialized in raw pixel space and post-step clamping is skipped, as described above.

5.2.4 Pretraining

VGG and ResNet experiments are each run under two network weight regimes: randomly initialized weights and ImageNet-pretrained weights. For torchvision backbone architectures, pretrained weights are the default ImageNet checkpoints provided by torchvision. When pretrained weights are used, the network is evaluated using ImageNet normalization statistics in place of dataset-specific statistics, since the pretrained weights were learned under those statistics. LeNet experiments use randomly initialized weights only. Running each configuration under both regimes allows assessment of whether gradient inversion success is sensitive to the network’s initial weights.

5.2.5 Early Stopping

Early stopping terminates optimization when the gradient matching loss falls below a threshold. This is reliable in standard neural network training, where the loss directly measures prediction error. Gradient inversion is different on two counts. First, the objective is non-convex (Section 4.6), so a stable loss may simply mean the optimizer has plateaued at a local minimum rather than converged to a good solution. Second, gradient matching is an imperfect proxy for image similarity (Section 4.7): a low loss provides no guarantee that the reconstruction resembles the true input.

A convergence threshold of 10^{-6} is nonetheless retained as a safeguard: in the case of a perfect or near-perfect reconstruction the gradient matching loss approaches zero, and terminating at that point avoids unnecessary further iterations. In practice, this threshold is never reached and every run completes the full iteration budget.

5.2.6 Restarts

Since the gradient matching objective is non-convex, optimization may converge to poor local minima. The restart count N specifies how many independent reconstructions are performed for a given configuration, each initialized from a different random seed. The reconstruction achieving the lowest gradient matching loss is retained. All ablation experiments use $N = 1$, corresponding to a single reconstruction without restarts. The restart analysis additionally evaluates $N \in \{1, 2, 3, 4, 5\}$ to investigate whether multiple restarts improve reconstruction quality.

5.2.7 iDLG Configuration

The iDLG configuration uses the L2 gradient matching loss as defined in Section 4.7. The optimizer is L-BFGS with 300 outer iterations, learning rate $\eta = 1$, 20 inner iterations per step, and a curvature history of 100 vector pairs. A closure is passed at each inner iteration to recompute the loss and gradient.

5.2.8 Inverting Gradients Configuration

The Inverting Gradients configuration uses the cosine-similarity loss as defined in Section 4.7, computed over all selected gradient entries concatenated into a single vector. The optimizer follows Geiping et al. [4]: AdamW with a signed gradient update, where the gradient with respect to the dummy image is replaced element-wise by its sign before each optimizer step:

$$g \leftarrow \text{sign}(\nabla_{\hat{x}} \mathcal{L}_{\text{cos}}). \quad (5.3)$$

AdamW weight decay is $\lambda = 10^{-5}$ and the initial learning rate $\eta = 0.1$. A multi-step schedule decays η by a factor of $\gamma = 0.5$ at iterations $\lfloor 3T/8 \rfloor$, $\lfloor 5T/8 \rfloor$, and $\lfloor 7T/8 \rfloor$, where $T = 5,000$ is the total iteration budget. Each optimizer step is a single gradient computation, whereas each L-BFGS outer step performs up to 20 inner function evaluations. Therefore, 300 L-BFGS outer iterations correspond to up to 6 000 gradient computations. The two budgets are approximately equal in total computational effort, so performance differences reflect the objective and optimizer choice rather than the iteration count.

5.2.9 Hyperparameter Summary

Table 5.1 consolidates the hyperparameters used across all reconstruction experiments. Shared settings apply to both setups, while the configuration-specific rows correspond to the iDLG and Inverting Gradients configurations described above.

	Hyperparameter	Value
iDLG	Gradient matching loss	L2
	Optimizer	L-BFGS
	Outer iterations	300
	Learning rate η	1
Inverting Gradients	Gradient matching loss	cosine-similarity
	Optimizer	Signed AdamW
	Iterations	5,000
	Learning rate η	0.1
	Weight decay λ	10^{-5}
	Learning-rate schedule γ	0.5

Table 5.1. Hyperparameters used in the reconstruction experiments. Shared settings apply to both configurations; the configuration-specific rows follow the iDLG and Inverting Gradients setups.

5.2.10 Label Inference

Both configurations recover the ground-truth label analytically using the iDLG procedure of Section 4.2, rather than optimizing a dummy label as in DLG. Following the exact implementation provided in the original iDLG codebase, the label is inferred once from the complete unmasked gradient by identifying the class whose final fully connected weight gradient yields a non-positive dot product with the gradient vectors of all other classes (as detailed in Algorithm 2).

The recovered class is then fixed and reused for the baseline and masked runs on that image, so that any difference in reconstruction quality is attributable to masking alone rather than to a different label being recovered. Because the inference is exact under the conditions of Section 4.2, the recovered label matches the ground truth in all experiments, and label error is therefore not a confounding factor in the paired comparison of Section 5.4.

5.2.11 Gradient Masking

The layer-wise ablation experiments use the `prefix_topfrac_entries_layer` mode: for each named layer group, gradient entries are ranked by absolute magnitude and the top fraction is retained. The retention fraction is set to 1.0 for every included group, so masking is binary at the layer-group level. A group is either fully retained or fully excluded from the shared gradient.

Each network architecture is partitioned into named layer groups. The baseline configuration retains all layer groups. Each masked configuration excludes one group, forming a leave-one-layer-out ablation that identifies which layer groups contribute most to reconstruction quality. Each layer-wise ablation is evaluated on 30 CIFAR-100 samples and repeated across the four configurations formed by the two optimizer/loss setups (Table 5.1) and the two weight regimes (randomly initialized and ImageNet-pretrained); LeNet is the exception, run only with randomly initialized weights.

The masking-percentage sweep instead selects the retained entries automatically by magnitude, without naming layers in advance. It uses the `gradsize_topfrac_entries_layer` mode, which takes a target fraction f as input and, within each parameter tensor (weight and bias ranked separately), keeps the f fraction of entries with the largest absolute gradient. The sweep varies f across runs to control how much of the gradient is retained, and is evaluated on 100 CIFAR-100 samples.

5.3 Implementation of Evaluation Metrics

Three metrics are used to evaluate reconstruction quality: MSE, PSNR, and SSIM. All are computed on unnormalized pixel values in $[0, 1]$, with the dummy image converted back to pixel space at each checkpoint via $x' = \hat{x}' \cdot \sigma + \mu$ and clamped to $[0, 1]$ as described in equation (5.2), while the ground-truth image is already in $[0, 1]$.

Mean squared error

MSE is computed at every iteration as

$$\text{MSE} = \frac{1}{C \cdot H \cdot W} \sum_{c,h,w} (x'_{chw} - x_{chw})^2, \quad (5.4)$$

where C, H, W are the number of channels, height, and width. The MSE reported for each experiment is taken from the checkpoint with the lowest gradient matching loss, rather than the MSE minimum across all iterations, as isolating the absolute minimum is unrealistic in practice. These can differ: the optimizer minimizes gradient loss, not pixel error, so the lowest loss reconstruction is not always the one with the lowest MSE.

Peak signal-to-noise ratio

PSNR is computed from the MSE at the checkpoint with the lowest gradient matching loss as

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right) \quad \text{dB}, \quad (5.5)$$

with $\text{MAX} = 1.0$ for pixel values in $[0, 1]$. Because PSNR is logarithmic, percentage improvements are reported as the equivalent reduction in MSE rather than as a percentage increase in PSNR; for a PSNR gain of Δ dB, this reduction is computed as $1 - 10^{-\Delta/10}$. A guard $\varepsilon = 10^{-12}$ is added to the denominator; MSE below this threshold returns $+\infty$.

Structural similarity index

SSIM is computed once per experiment at the same checkpoint with the lowest gradient matching loss, using scikit-image's `structural_similarity` with a 7×7 window, across all three channels jointly. The stabilization constants $C_1 = (K_1 L)^2$ and $C_2 = (K_2 L)^2$ defined in Section 4.10.3 use the scikit-image default coefficients $K_1 = 0.01$ and $K_2 = 0.03$ [23]; with $L = 1$ for pixel values in $[0, 1]$ this gives $C_1 = 10^{-4}$ and $C_2 = 9 \times 10^{-4}$. Scores are averaged when the batch has more than one image.

5.4 Paired statistical comparison of reconstruction quality

Each masked configuration is compared against its baseline within a paired design. Because reconstruction quality varies widely across images, the per-image differences are not guaranteed to be normally distributed; their distribution is therefore assessed explicitly (Section 4.10.4) before any parametric summary is used, and reconstruction quality is otherwise reported with descriptive statistics.

Paired design

Each experiment draws a fixed ground-truth image, computes its gradient on a freshly initialized network, and runs both the baseline and masked reconstructions under the same conditions. This gives one PSNR value per method per image. The quantity of interest is the per-image difference:

$$d_i = \text{PSNR}_i^{\text{masked}} - \text{PSNR}_i^{\text{baseline}}, \quad i = 1, \dots, n. \quad (5.6)$$

Pairing on the same image removes the between-image variation in reconstruction difficulty and isolates the effect of masking.

Normality assessment

Whether the per-image differences d_i can be summarized using parametric statistics depends on their underlying distribution. Because reconstruction difficulty varies substantially across images, the differences cannot be assumed normal in advance, and normality is therefore assessed rather than presumed. The Shapiro-Wilk test is applied to the PSNR, MSE, and SSIM differences for each experimental configuration at the $\alpha = 0.05$ significance level, and the resulting per-condition p -values are tabulated in Appendix C.

Which metric is supplied to the test matters, because the three reconstruction metrics behave very differently under this assumption. Across images, reconstruction difficulty varies widely and MSE is bounded below by zero, so the per-image MSE values form a right-skewed distribution with a long upper tail. PSNR applies the strictly concave $\log_{10}$ to this quantity, compressing large values more than small ones; the upper tail is drawn inward and the distribution made more symmetric. Differences in PSNR between ground-truth and reconstructed images may therefore satisfy the normality assumption of the paired t -test more reliably than differences in MSE.

Beyond the shape of the differences, both the Shapiro-Wilk test and the paired t -test require the observations themselves to be independent. This condition is met by construction: each difference d_i is obtained from a reconstruction on a distinct ground-truth image with an independently initialized network, so $d_1, \dots, d_n$ are naturally independent.

This chapter reports reconstruction quality for each architecture and dataset, comparing baseline gradient inversion against masked variants across the experimental configurations described in Chapter 5. Results are presented for LeNet, VGG-11, VGG-13, and ResNet-18; a qualitative summary for ResNet-50 is given in Section 6.4.2 and full per-layer statistics in Appendix C.2.2, as it produces a similar pattern to ResNet-18.

6.1 Normality tests

Normality tests were performed for all results following the procedure described in Section 5.4. Because normality is rejected for almost all configurations, the distributional assumption underlying parametric inference such as the paired t -test does not hold. Consequently, the results in this chapter are reported using descriptive statistics, principally the sample mean and standard deviation. Additionally, box and violin plots are used to visualize per-image distributions, characterizing the magnitude and spread of reconstruction differences without assuming a specific underlying distribution.

6.2 LeNet

The ablation evaluates four named layer groups on LeNet with CIFAR-100: `body.0`, `body.2`, and `body.4` (convolutional layers) and `fc` (fully connected layer). Each configuration withholds one group while retaining the other three.

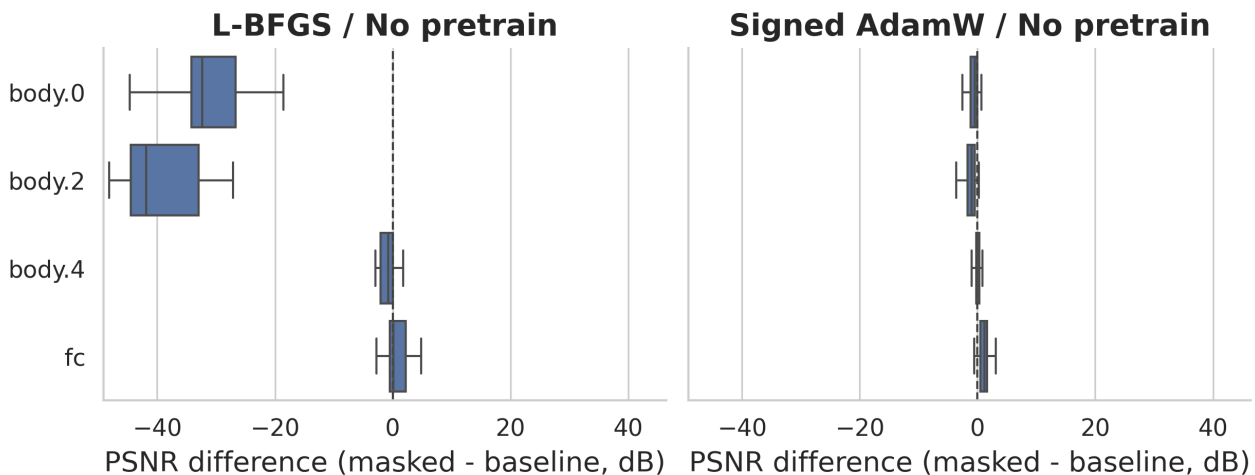


Figure 6.1. Layer-wise masking ablation for LeNet on CIFAR-100. Each box plot shows the per-sample PSNR difference between the masked reconstruction and the unmasked baseline; the dashed vertical line marks no change. Negative values indicate lower reconstruction quality after masking.

The LeNet results show a clear contrast between masking convolutional layers and masking the fully connected layer. Under L-BFGS with L2 loss, masking early convolutional layers gives the largest reductions: mean PSNR drops by 27.40 dB for `body.0` and by 35.14 dB for `body.2`; masking `body.4`, the final convolutional layer, produces a smaller reduction (40.98 $\rightarrow$ 37.88 dB, -3.10 dB). In contrast,

masking `fc` increases the mean from 40.98 to 45.49 dB (+4.51 dB). Full metric values for all LeNet configurations are reported in Appendix Table C.1.

Under Signed AdamW with cosine-similarity loss, the same broad ordering holds but the changes are smaller. Masking `body.2` gives the clearest reduction (-1.32 dB) and `body.0` a smaller one (-0.82 dB), while `body.4` is nearly neutral ($9.78 \rightarrow 9.89$ dB, $+0.11$ dB). Masking `fc` gives the largest increase, from 9.78 to 10.85 dB ($+1.07$ dB). Overall, cosine-similarity produces substantially worse reconstructions than L2: all of its configurations stay near 9–11 dB, far below the 38–45 dB range obtained under L-BFGS with L2 loss.

6.3 VGG

Layer-wise ablations were conducted on both VGG-11 and VGG-13. In all VGG ablation runs, the classifier gradients are fully masked; rows labeled by feature layers therefore show the effect of additionally masking the listed feature layer. Both architectures exhibit the same broad pattern: masking benefits reconstruction without pretraining but is harmful once pretrained weights are used.

6.3.1 VGG-11

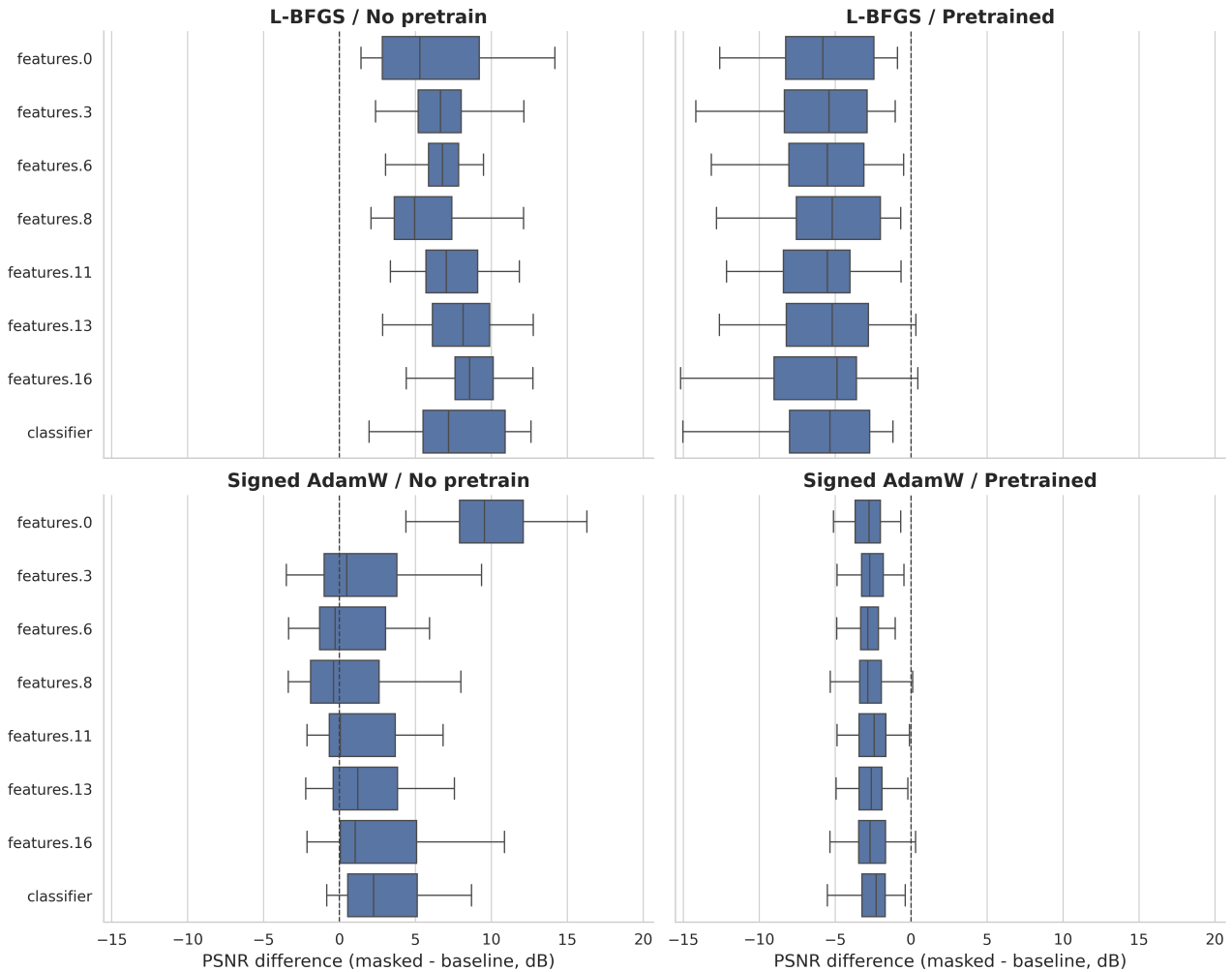


Figure 6.2. Layer-wise masking ablation for VGG-11 on CIFAR-100 across optimizer/loss and pretraining settings. Each box plot shows the per-sample PSNR difference between the masked reconstruction and the unmasked baseline; the dashed vertical line marks no change. The classifier is fully masked in all configurations, and feature rows additionally mask the listed feature layer. Without pretraining the distributions are predominantly positive; with pretrained weights they shift left, indicating lower reconstruction quality after masking.

The VGG-11 ablation shows that masking is beneficial in the non-pretrained setting but harmful once pretrained weights are used. Without pretraining, every evaluated layer mask increases the mean PSNR: under L-BFGS it rises from a baseline of 13.86 dB to between 19.66 and 22.53 dB, and under Signed AdamW from 20.57 dB to between 21.74 and 31.44 dB, with mean SSIM rising by 52% to near-exact reconstruction quality for the best configuration (Table C.4). With pretrained weights the trend reverses: under L-BFGS, masking reduces the mean from 14.44 dB to approximately 8.4–9.1 dB, while under Signed AdamW the baseline of 11.50 dB falls to between 8.69 and 8.97 dB. Per-layer statistics are given in Table C.4.

Masking the classifier together with `features.0` gives the largest of these gains and recovers VGG-11 inputs almost perfectly under Signed AdamW. Relative to the unmasked baseline, mean PSNR rises by 10.9 dB (to 31.44 dB), mean MSE falls by 94% (from 0.016 to 0.0009), and mean SSIM rises by 52% (from 0.62 to 0.94), leaving reconstructions very close to the ground-truth images both visually and numerically (Figures D.2 and D.1).

Because this configuration already recovers VGG-11 inputs almost perfectly, it leaves little room to extend masking to multi-layer configurations with fractional per-layer retention. The remaining VGG analysis therefore focuses on VGG-13: as a deeper network it is harder to invert and closer to the architectures used in practice, while still allowing investigation of whether the same masking trends carry over.

6.3.2 VGG-13

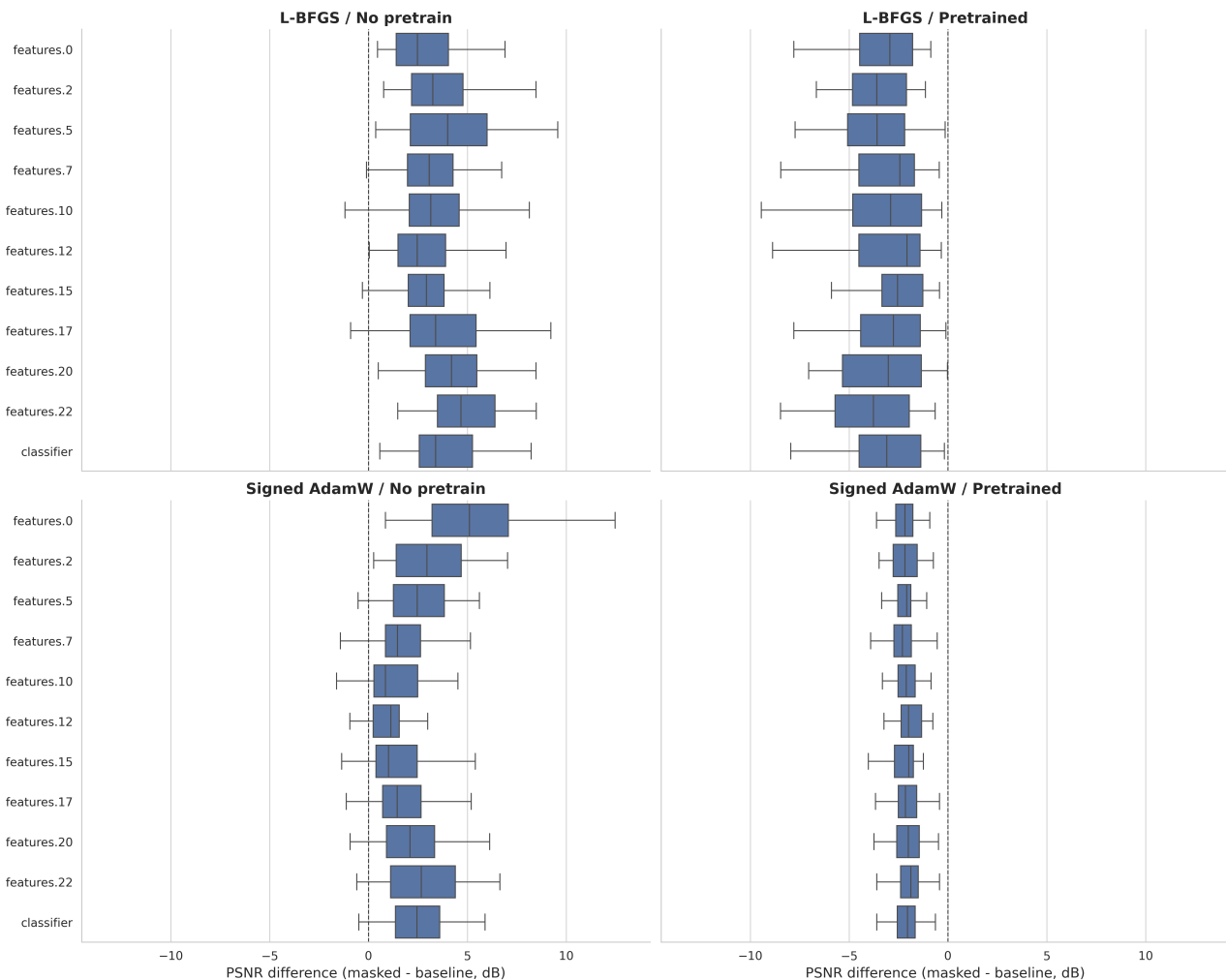


Figure 6.3. Layer-wise masking ablation for VGG-13 on CIFAR-100 across optimizer/loss and pretraining settings. Each box plot shows the per-sample PSNR difference between the masked reconstruction and the unmasked baseline; the dashed vertical line marks no change. The classifier is fully masked in all configurations, and feature rows additionally mask the listed feature layer.

Without pretraining, every evaluated mask raises the mean PSNR above the unmasked baseline under both optimizers. Under Signed AdamW with cosine-similarity loss, the baseline of 15.81 dB increases to 21.26 dB when `features.0` is masked, the largest value in this setting; the per-layer means then fall toward the middle of the network before rising again at the deepest layers, ranging from 17.03 to 19.07 dB across the remaining feature layers. Under L-BFGS with L2 loss the baseline is 12.61 dB and the per-layer means range from 15.67 to 17.60 dB, peaking at `features.22`.

With pretrained weights the direction reverses: every mask lowers the mean PSNR below the baseline, to between 8.38 and 9.69 dB against 12.35 dB under L-BFGS, and to between 8.15 and 8.53 dB against 10.41 dB under Signed AdamW. Full metric values for all configurations are reported in Appendix Table C.5.

Because the single-layer ablation showed that feature layers improve reconstruction by clearly different amounts, the next experiments combined masking across multiple feature layers and varied the retained fraction within each selected layer, to test whether a tuned combination improves reconstruction further.

Best masking configuration

Based on the layer-wise trends observed in Figure 6.3 and subsequent experiments, a final masking configuration was selected. Layers that produced the largest reconstruction improvements when masked, namely the first convolutional layer, the final convolutional layer, and the classifier layers, were removed entirely. While 50% of the second and second-to-last convolutional layers were masked, and all remaining convolutional layers were assigned a masking level of 10% to suppress low magnitude gradient entries without substantially altering the optimization signal. The resulting configuration is shown in Table 6.1.

Layer	Masked (%)	Layer	Masked (%)
<code>features.0</code>	100	<code>features.17</code>	10
<code>features.2</code>	50	<code>features.20</code>	50
<code>features.5</code>	10	<code>features.22</code>	100
<code>features.7</code>	10	<code>classifier.0</code>	100
<code>features.10</code>	10	<code>classifier.3</code>	100
<code>features.12</code>	10	<code>classifier.6</code>	100
<code>features.15</code>	10		

Table 6.1. Selected VGG-13 masking configuration. Fractions indicate the percentage of gradient entries masked within each layer after ranking entries by absolute magnitude.

Dataset	Method	Avg. MSE	Avg. PSNR	Avg. SSIM	MSE < 0.01	MSE diff.	SSIM diff.
CIFAR-100	Unmasked	0.0330	15.81	0.377	3/50	—	—
CIFAR-100	Masked	0.0073	23.14	0.751	39/50	−78%	99%
LFW	Unmasked	0.0452	14.14	0.345	1/50	—	—
LFW	Masked	0.0085	22.50	0.816	35/50	−81%	137%

Table 6.2. Effect of the selected VGG-13 gradient-masking configuration on CIFAR-100 and LFW after 5000 reconstruction iterations, evaluated on 50 images per dataset. Masking improves reconstruction quality on both datasets, lowering average MSE, increasing average PSNR and SSIM, and increasing the number of reconstructions below the MSE threshold of 0.01. Percentage changes are computed relative to the corresponding unmasked baseline.

The selected configuration improves reconstruction quality on both datasets after 5,000 iterations. The largest mean gain is observed on LFW, where PSNR increases by 8.36 dB, while CIFAR-100 shows a similar increase of 7.33 dB. The improvement is also reflected in the threshold counts: the number of reconstructions with MSE below 0.01 increases from 3 to 39 on CIFAR-100 and from 1 to 35 on LFW, with mean SSIM rising by 99% and 137% respectively.

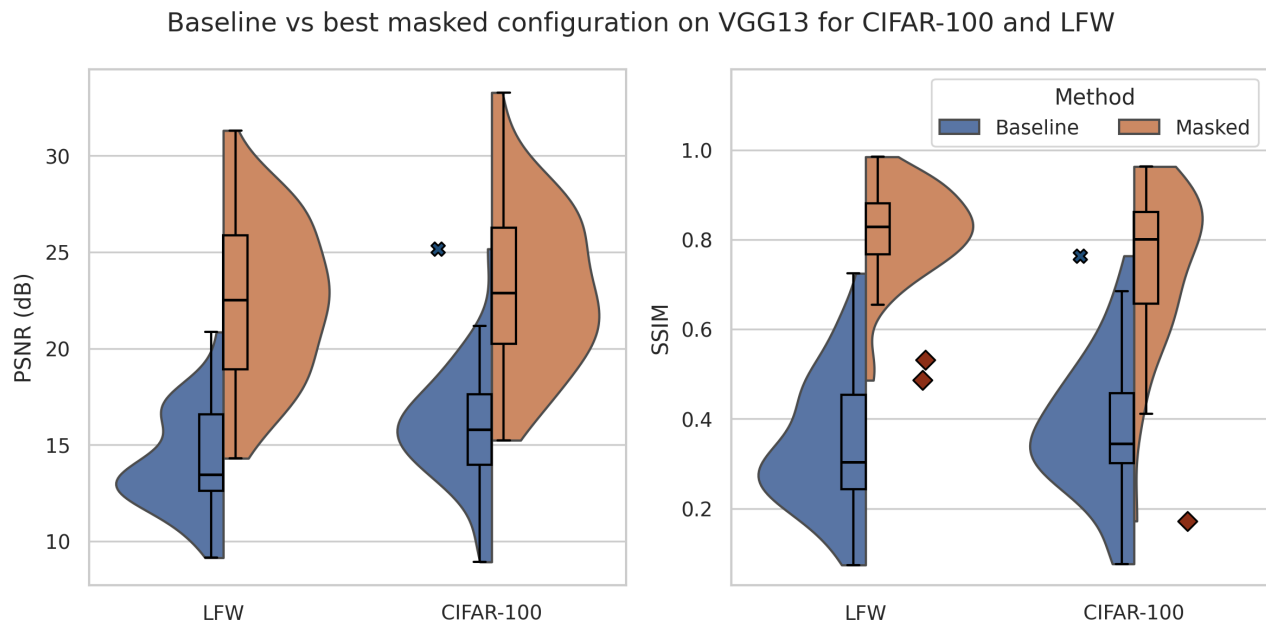


Figure 6.4. Reconstruction quality for the unmasked baseline and selected VGG-13 masking configuration on LFW and CIFAR-100. Violin plots summarize PSNR and SSIM across 50 reconstructions per dataset, with embedded box plots showing the median and interquartile range (IQR). Higher values indicate better reconstruction quality; masking shifts both metrics upward on both datasets. Crosses denote outliers for the baseline while diamonds are outliers for the masked method.

The distributional shift in Figure 6.4 is consistent across PSNR and SSIM, with the masked reconstructions moving toward higher values on both datasets. Figure 6.5 further shows that the selected mask improves both the strongest and weakest paired examples, although the magnitude of the improvement varies substantially across samples.

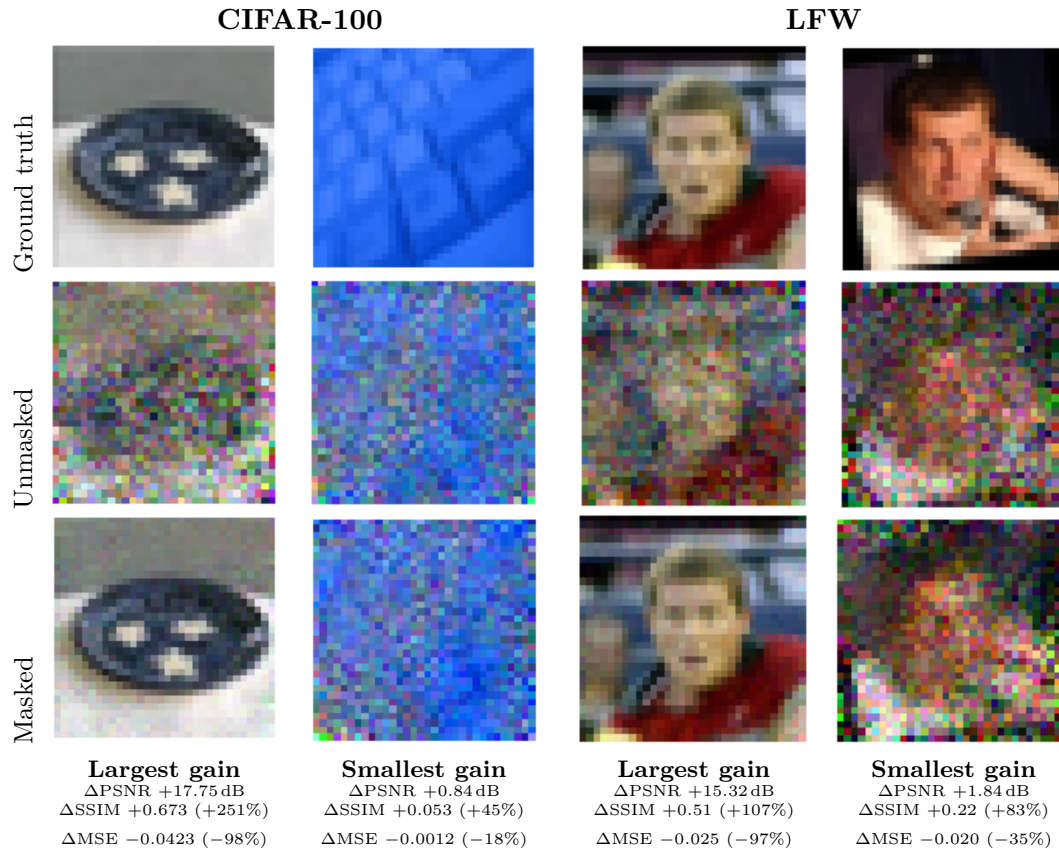


Figure 6.5. Representative examples of gradient inversion reconstructions before and after gradient masking using the best VGG-13 configuration on CIFAR-100 and LFW. For each dataset, the columns show the samples with the largest and smallest PSNR improvement after masking. Rows correspond to the ground truth image, the reconstruction obtained from the full gradient, and the reconstruction obtained after masking. The reported Δ values are computed relative to the unmasked reconstruction; positive ΔPSNR and ΔSSIM , and negative ΔMSE , indicate improved reconstruction quality.

6.4 ResNet

The ablation is conducted on two architectures, ResNet-18 and ResNet-50, evaluating the same named layer groups (`conv1`, `bn1`, `layer1–layer4`, and `fc`). Both architectures produce the same qualitative pattern; ResNet-18 results are presented below and a qualitative summary for ResNet-50 is given in Section 6.4.2, with full per-layer statistics in Appendix C.2.2.

6.4.1 ResNet-18

ResNet-18 uses basic residual blocks, with each row withholding one named layer group while retaining all others.

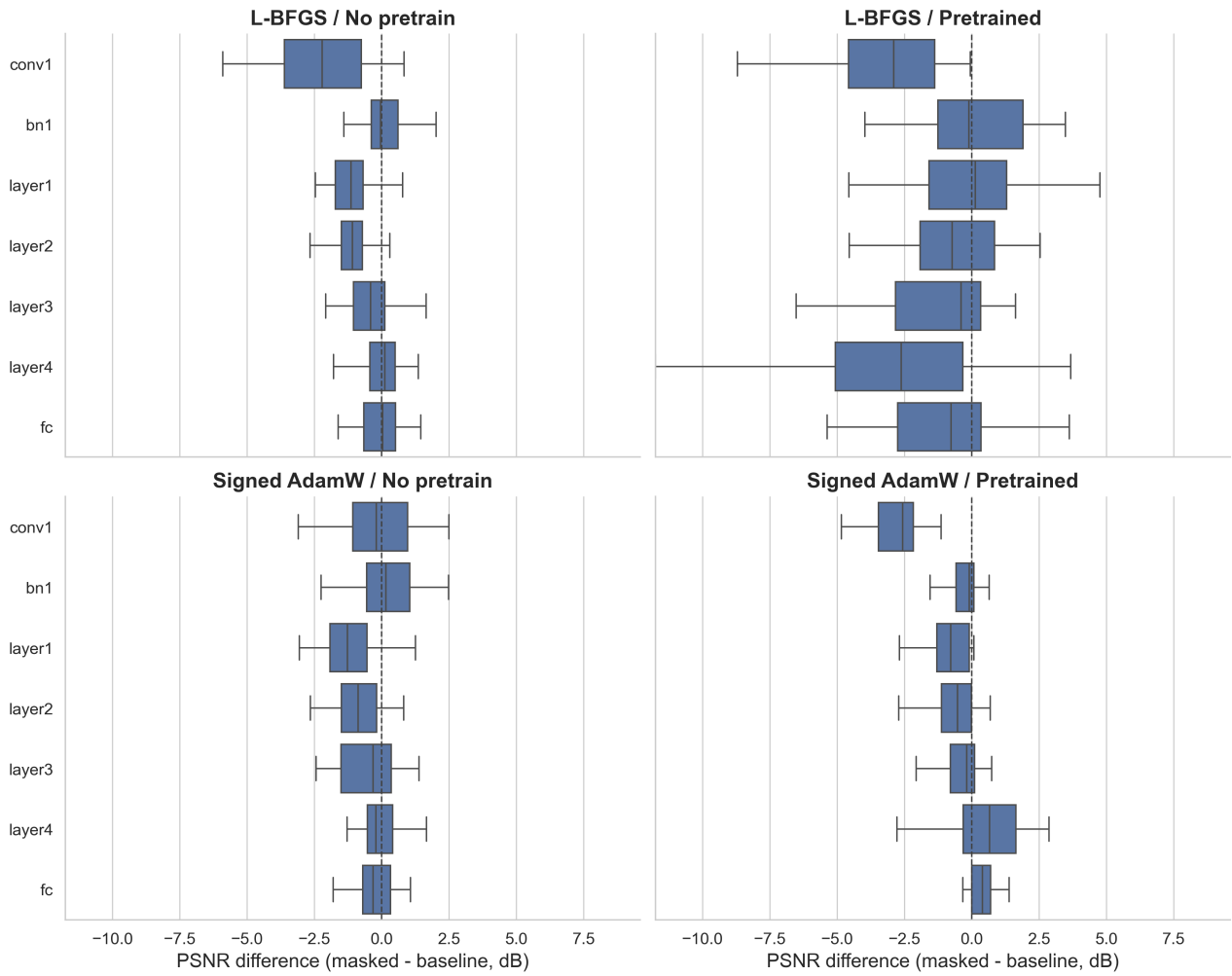


Figure 6.6. Layer-wise masking ablation for ResNet-18 on CIFAR-100 across optimizer/loss and pretraining settings. Each box plot shows the per-sample PSNR difference between the masked reconstruction and the unmasked baseline; the dashed vertical line marks no change. Negative values indicate lower reconstruction quality after masking.

`conv1` is the most damaging layer to mask in three of the four configurations. From the unmasked baseline of 17.53 dB, mean PSNR falls to 15.23 dB and mean SSIM by 41% ($0.54 \rightarrow 0.32$) under L-BFGS without pretraining; the drop is larger with pretraining, reaching $17.46 \rightarrow 14.41$ dB under L-BFGS and $15.05 \rightarrow 12.31$ dB under Signed AdamW. The exception is cosine-similarity without pretraining, where masking `conv1` barely changes reconstruction. The early residual groups (`layer1`–`layer3`) are consistently harmful to mask for non-pretrained networks regardless of optimizer, while `bn1` stays close to the baseline throughout. Full metric values are reported in Appendix Table C.2.

With pretrained weights, sensitivity spreads across depth: under L-BFGS, masking `layer3` or `layer4` also lowers quality (16.16 and 14.58 dB), whereas `bn1` and `layer1` remain near baseline. The one consistently positive effect is `layer4` under pretrained Signed AdamW, where masking raises the mean slightly above baseline ($15.05 \rightarrow 15.43$ dB), suggesting its gradients carry little recoverable signal once pretrained weights are used.

6.4.2 ResNet-50

ResNet-50 uses bottleneck residual blocks and produces the same qualitative pattern as ResNet-18: in the non-pretrained setting `conv1` is the most damaging layer to mask and the remaining groups stay close to baseline; with pretrained weights, middle layers yield slight improvements while the `fc` layer degrades reconstruction under both optimizers. Full per-layer statistics are provided in Appendix C.2.2.

6.5 Jacobian Rank

The two entry selection modes `layer_spread` and `topk_abs` are compared (Section 5.1). The minimum entry count at which full rank ($d = 3,072$) was first achieved across all 30 samples is reported for each architecture on CIFAR-100; full numerical results are given in Appendix B.1.

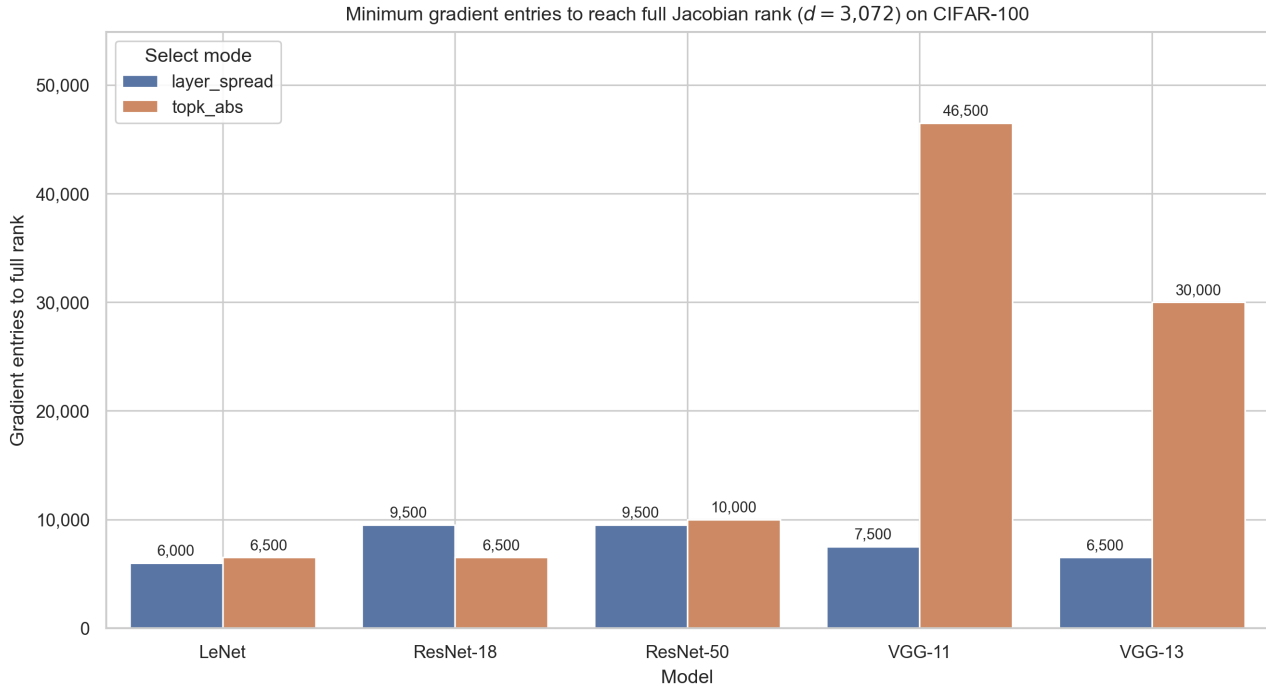


Figure 6.7. Minimum number of gradient entries required to reach full Jacobian rank ($d = 3,072$) per architecture and selection mode on CIFAR-100, evaluated over 30 samples.

Under `layer_spread`, full rank is achieved consistently across all architectures within a tight range of 6,000–9,500 entries, showing little sensitivity to network depth or size. Under `topk_abs`, results vary substantially: ResNet-18 and ResNet-50 reach full rank at 6,500 and 10,000 entries respectively, whereas VGG-13 requires 30,000 and VGG-11 requires 46,500. For most architectures, `layer_spread` requires fewer entries to reach full rank than `topk_abs`; ResNet-18 is the exception, where `topk_abs` achieves full rank at 6,500 entries against 9,500 under `layer_spread`. In both cases the entry counts represent a small fraction of each architecture’s total gradient vector: 46,500 entries for VGG-11 under `topk_abs` is less than 0.04% of that network’s ≈ 129 M gradient parameters, and 9,500 entries for ResNet-18 under `layer_spread` is less than 0.1% of its ≈ 11.5 M.

6.5.1 Rank Reconstruction Sweep

The rank sweep establishes when full rank is achieved; this experiment examines whether full rank is also sufficient for reconstruction. A single image is reconstructed at increasing numbers of retained entries under two entry pool configurations: FC included and FC excluded from the gradient matching objective.

The figures use `layer_spread` entry selection; this matches one of the two Jacobian rank sweep modes, so the rank values correspond directly to those in Figure 6.7. The other two modes, `topk_abs` and `gradsize_topfrac_entries_layer`, are in Appendix B.1.

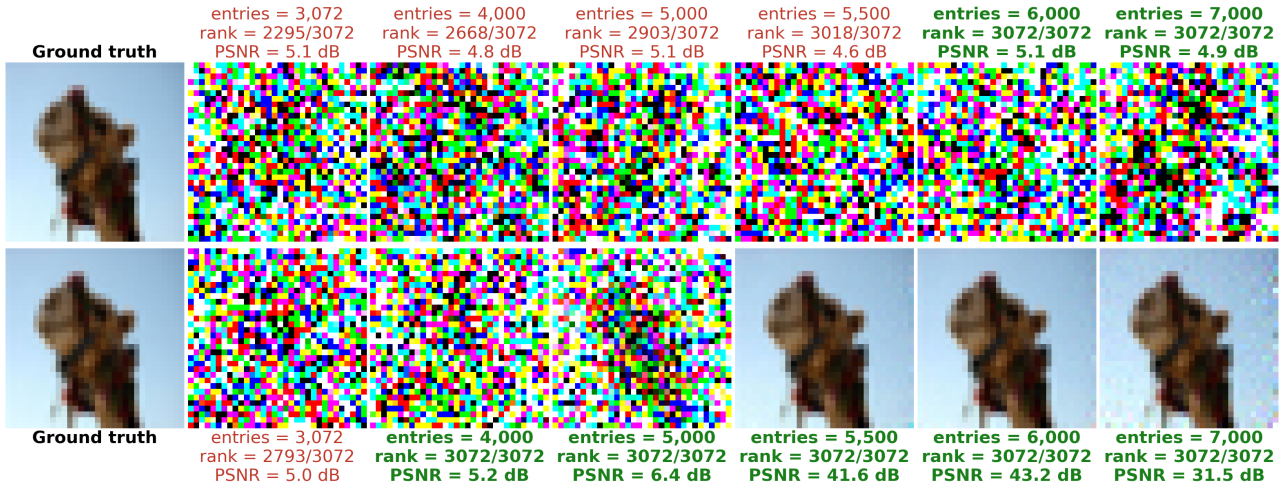


Figure 6.8. Rank reconstruction sweep using `layer_spread` entry selection on LeNet / CIFAR-100. Top: FC layer included in the entry pool; bottom: FC layer excluded. Labels above each top panel and below each bottom panel show retained entries, Jacobian rank, and PSNR. Green (bold) = full rank (rank = $d = 3,072$); red = rank deficit.

Figure 6.8 shows two patterns. First, excluding the FC layer reduces the number of entries needed to reach full rank: with FC excluded, full rank is achieved at 4,000 entries compared to 6,000 entries when the FC layer is included. Second, reconstruction quality is substantially higher at every entry count when the FC layer is excluded: PSNR rises to 41.6dB at 5,500 entries and 43.2dB at 6,000 entries, while the FC-included configuration stays at approximately 5 dB even after achieving full rank at 6,000 and 7,000 entries.

The same two patterns appear under `topk_abs` selection (Figure B.1, Appendix B.1), where the FC-included configuration never reaches full rank within the tested range. Whereas in `gradsize_topfrac_entries_layer` (Figure B.2, Appendix B.1), both configurations reach full rank at the same threshold (`topfrac = 0.7`), but reconstruction quality diverges above it: PSNR with FC excluded reaches 40.8dB at `topfrac = 0.8` and 43.7dB at `topfrac = 1.0`, while the FC-included configuration does not exceed 5.0dB. Since `gradsize_topfrac_entries_layer` is the same selection mode used in the masking-percentage sweep below, these PSNR values can be read alongside the reconstruction counts in Figure 6.9.

6.6 Number of images reconstructed vs masking percentage

This sweep varies the fraction of gradient entries masked and counts how many of 100 images are successfully reconstructed ($MSE < 0.01$) for LeNet, ResNet-18, and VGG-13 on CIFAR-100, using `gradsize_topfrac_entries_layer` entry selection.

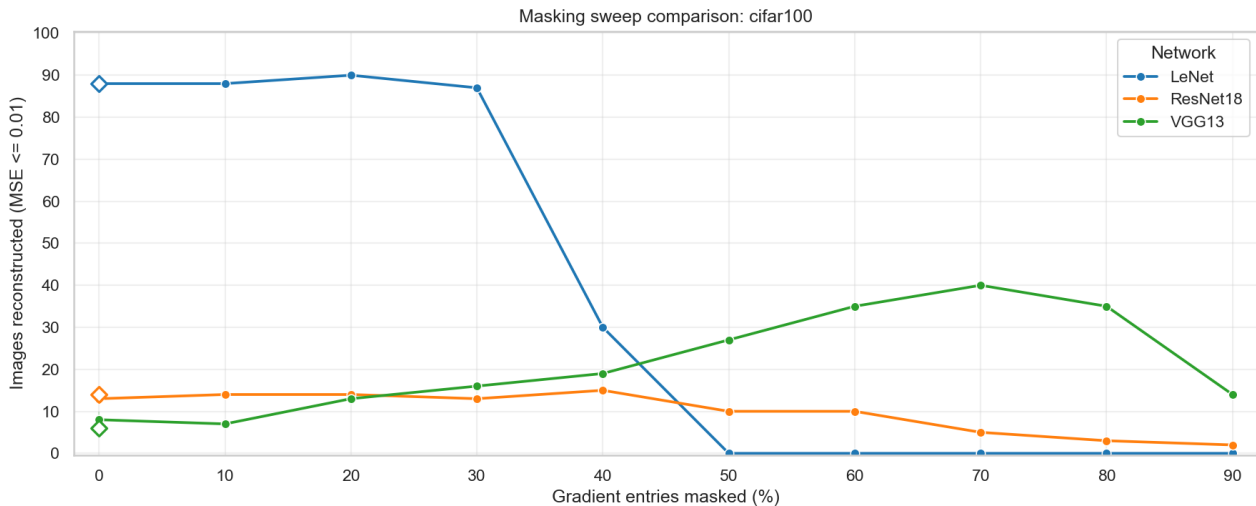


Figure 6.9. Masking-percentage sweep on CIFAR-100 for LeNet, ResNet-18, and VGG-13. The y-axis counts how many of 100 images reach $\text{MSE} < 0.01$; the x-axis gives the percentage of gradient entries masked, and diamonds mark unmasked baselines.

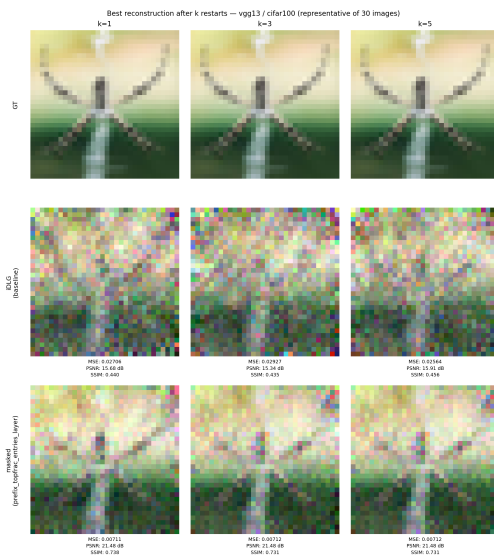
The masking sweep shows distinct architecture-dependent trends. LeNet reconstructs the largest number of images without substantial masking, reaching approximately 90 successful reconstructions at 20% masking, but this drops rapidly after 30% and reaches zero from 50% masking onward. ResNet-18 remains low across the sweep, with only a small peak of 15 successful reconstructions at 40% masking. VGG-13 shows the opposite pattern to LeNet: the number of successful reconstructions increases with stronger masking up to 70%, where it reaches 40 out of 100 images, before declining at higher masking percentages.

6.7 Effect of Restarts

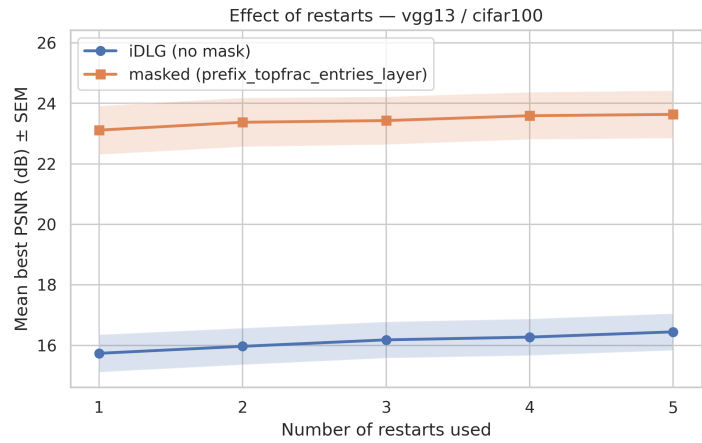
All preceding experiments used a single optimization run from one random initialization. This experiment instead repeats each reconstruction from k independent random initializations and keeps the run with the lowest gradient matching loss, increasing k from 1 to 5. Figures 6.10 and 6.11 report the effect on CIFAR-100 for VGG-13 and ResNet-18. Each figure pairs one example image’s reconstructions at $k = 1, 3$, and 5 (left) with the mean PSNR of the kept run across all 30 images as a function of k (right); a higher curve means better reconstruction.

6.7.1 VGG-13

For VGG-13, additional restarts improve both the unmasked and masked reconstructions, but the masked configuration remains consistently higher across the full range. From $k = 1$ to $k = 5$, the unmasked baseline increases from 15.77 to 16.53 dB, whereas the masked configuration increases from 23.11 to 23.63 dB. The gap therefore narrows slightly, from approximately 7.34 to 7.10 dB.



(a) Reconstructions of a single CIFAR-100 image, chosen as the image with the median best PSNR at $k = 5$ so that it represents a typical case rather than a best or worst one. Columns show the kept reconstruction (the restart with the lowest gradient matching loss) after $k = 1, 3$, and 5 restarts; rows show the ground truth, the unmasked baseline reconstruction, and the masked configuration.



(b) Mean best PSNR over 30 CIFAR-100 images as the number of restarts increases. Shaded bands show $\pm$ one standard error of the mean.

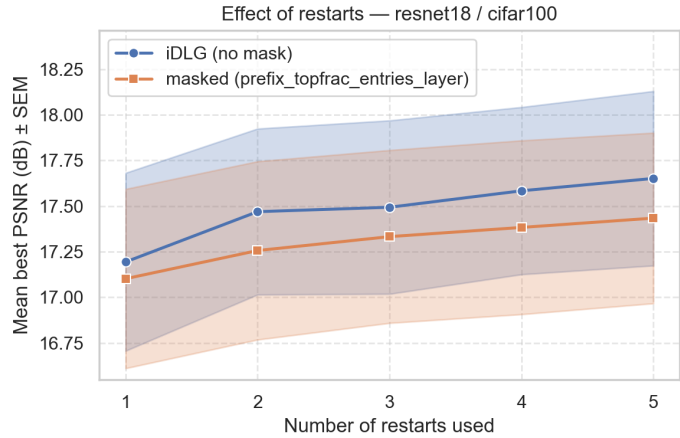
Figure 6.10. Effect of random restarts on VGG-13 reconstruction quality for CIFAR-100, combining the reconstruction sequence for the median image (a) with the mean best-PSNR trend over all 30 images (b).

6.7.2 ResNet-18

For ResNet-18, the effect of restarts is smaller and the ordering is reversed. The unmasked baseline increases from 17.19 to 17.65 dB between $k = 1$ and $k = 5$, while the masked configuration increases from 17.10 to 17.43 dB. The unmasked curve remains slightly higher throughout, with the gap increasing from approximately 0.09 to 0.22 dB.



(a) Reconstructions of a single CIFAR-100 image for ResNet-18, chosen as the image with the median best PSNR at $k = 5$ so that it represents a typical case rather than a best or worst one. Columns show the kept reconstruction (the restart with the lowest gradient matching loss) after $k = 1, 3$, and 5 restarts; rows show the ground truth, the unmasked baseline reconstruction, and the masked configuration.



(b) Mean best PSNR over 30 CIFAR-100 images as the number of restarts increases. Shaded bands show $\pm$ one standard error of the mean.

Figure 6.11. Effect of random restarts on ResNet-18 reconstruction quality for CIFAR-100, combining the reconstruction sequence for the median image (a) with the mean best-PSNR trend over all 30 images (b).

The masking experiments reveal an architecture-dependent pattern where the outcome follows from how each network distributes gradient information across its layers, with the results for each architecture discussed in the sections below.

7.1 Interpretation of LeNet results

The LeNet ablation provides a shallow-network baseline for interpreting the masking results. In a purely sequential network with no skip connections, `body.0` and `body.2` process the input with nothing before them, so their gradients provide the primary pixel-level constraints in the matching objective. Masking either therefore removes most of that signal rather than reducing a redundant part of it, which is why the degradation is so severe: masking `body.2` alone drops PSNR from 40.98 to 5.84 dB and collapses SSIM by 97% under L-BFGS, masking `body.0` similarly degrades reconstruction. `body.4` is a different case: by that point the input has been transformed by two preceding layers, so its gradient encodes more abstract feature structure than raw pixel values. This depth-dependent pattern is consistent with earlier layers imposing the most direct constraints on the input, with each successive layer encoding progressively more abstract representations.

L-BFGS is therefore especially sensitive to losing `body.0` or `body.2`: exploiting the full magnitude of those pixel-level gradients is what enables it to achieve near-perfect reconstructions on LeNet, and removing them collapses that signal entirely. Cosine similarity with Signed AdamW was designed for a different setting (stabilizing inversion in deep networks where gradient magnitudes vary widely and Euclidean matching becomes numerically unstable [4]), and LeNet offers none of those challenges. Working with sign-only updates, it is already operating with a degraded signal, so losing one layer's gradient costs far less: the same intervention just mentioned that drops 27 dB under L-BFGS costs under 1 dB under Signed AdamW.

Masking `fc` improves reconstruction in both configurations, most strongly under L-BFGS (+4.51 dB) and more modestly under Signed AdamW (+1.07 dB), where mean SSIM nonetheless rises by 54% (0.13 → 0.20). This does not contradict the importance of the final layer for iDLG label recovery. The label is inferred analytically from the final-layer gradient before the reconstruction optimization (Section 4.2), so the `fc` gradient's remaining role in the matching objective is to constrain the dummy input through classifier logits rather than to identify the class. The `fc` gradient therefore contains high-level classifier constraints, while the convolutional gradients retain the more direct dependence on local image structure. Removing `fc` can consequently improve optimization by preventing these classifier-level constraints from dominating the gradient matching objective, while retaining the convolutional constraints that are most useful for reconstructing the image. This is the clearest case in these experiments where a gradient component carries information but still hinders reconstruction by dominating the objective.

The masking-percentage sweep (Figure 6.9) connects this layer-level picture to the quantity of gradient retained: LeNet reconstructs reliably under light masking but collapses sharply because beyond roughly 30 % masking the retained entries no longer span enough independent constraints to determine all image pixels, as the Jacobian analysis below shows.

7.1.1 Jacobian rank and reconstruction

The rank reconstruction sweep (Figure 6.8) reveals two effects of excluding the FC layer from the entry pool. First, full Jacobian rank is reached with fewer entries: excluding FC reduces the threshold from 6,000 to 4,000 under `layer_spread`, since convolutional entries are tied to specific spatial positions and channels providing more diverse coverage per entry than FC entries. Second, and more interestingly, achieving full rank is necessary but not sufficient: even at the entry count where both configurations reach full rank, reconstruction succeeds only with FC excluded (PSNR rises to 43.2 dB) while the FC-included configuration stays near 5 dB. A plausible explanation is numerical dominance: LeNet’s FC layer holds substantially more gradient entries than its convolutional layers combined, so including it drives the matching objective toward class-level gradient structure at the expense of the pixel-constraining convolutional terms, leaving the non-convex objective stuck at a poor solution even when the Jacobian is locally full rank. This pattern is consistent across all three selection modes (Figures B.1 and B.2, Appendix B.1).

This also explains the two causes of failure visible in the masking-percentage sweep. Under the `gradsize_topfrac` selection, the Jacobian retains full column rank until roughly 30% of entries are masked (Figure B.2). Light masking therefore stays in the range where the Jacobian remains full rank, and reconstruction quality is governed by optimization conditioning; beyond that threshold, the retained entries no longer span enough independent constraints to determine all image pixels, and failure is better attributed to missing information than to poor optimization. Since the rank threshold comes from a single image and the sweep averages over 100, the alignment at 30% should be read as qualitative rather than exact.

7.2 Interpretation of VGG results

Among the architectures studied, VGG provides the clearest evidence that poor reconstruction does not imply a lack of recoverable information. In both VGG-11 and VGG-13, masking improves reconstruction without pretraining and degrades it after pretraining.

7.2.1 Layer-wise masking pattern

Without pretraining, every evaluated mask improves reconstruction on both networks. On VGG-11 the effect is strongest under Signed AdamW with cosine similarity, where masking the classifier together with `features.0` raises mean PSNR from 20.57 to 31.44 dB (Table C.4), close enough to the ground truth that little room remains for further masking. VGG-13 follows the same direction from a weaker baseline: the largest single-layer gain again falls at `features.0`, gains shrink toward the middle of the feature extractor, then grow again at the deepest layers (Table C.5).

This matches the conditioning argument of Section 4.5: early-layer gradients carry direct input information but reach the layer through a long backpropagation path that, under random weights, leaves them poorly conditioned for pixel-level optimization, so masking them reduces interference without discarding the information they hold. The same argument explains the gap between the networks, since VGG-13’s two additional convolutional layers lengthen that path, consistent with its lower baseline (15.81 versus 20.57 dB) and the wider spread of its single-layer gains.

Which layer gains most from masking depends on the loss and optimizer. Under cosine similarity with Signed AdamW the strongest gain falls at `features.0`, because both the loss and the sign update discard gradient magnitude, so optimization turns on directional agreement and poorly conditioned directions dominate even when small. Under L-BFGS with L2 loss the largest gains occur deeper in the feature extractor, since L2 retains magnitudes and L-BFGS weights updates by local curvature [18]. The localization shifts with the optimizer, but the underlying pattern, that some gradients hinder rather than help, does not.

Varying how much of the gradient is masked rather than which layer confirms the same pattern and exposes its limit. On VGG-13 the number of images reconstructed with MSE below 0.01 rises from 6 at no masking to a peak of 40 near 70% masking, then declines (Figure 6.9). Removing poorly conditioned constraints helps up to a point; beyond it, masking discards constraints the reconstruction needs.

Effect of pretraining

Pretraining reverses the masking pattern on both networks: every intervention that helped without pretraining instead lowers reconstruction quality with it, including masking the classifier, which improves VGG-13 without pretraining (12.61 $\rightarrow$ 16.73 dB under L-BFGS, 15.81 $\rightarrow$ 18.57 dB under Signed AdamW) but lowers it once pretrained (12.35 $\rightarrow$ 9.29 and 10.41 $\rightarrow$ 8.40 dB; Table C.5). The same reversal appears in SSIM: without pretraining the classifier mask raises it by +50% under Signed AdamW (0.38 $\rightarrow$ 0.57), whereas with pretrained weights the same mask lowers it by -60% (0.10 $\rightarrow$ 0.04). Layer position alone therefore does not determine whether a gradient component should be removed. A plausible explanation follows the role of pretraining described in Section 4.4: in a randomly initialized network the early-layer and classifier gradients are poorly conditioned or impose class-level constraints that compete with the pixel-constraining convolutional gradients, so removing them helps, whereas after pretraining the same gradients reflect learned, input-relevant structure, so removing them discards useful constraints instead.

7.2.2 Multi-layer masking configuration

The best VGG-13 configuration masks early and late feature layers heavily while retaining the majority of middle layers, and discarding all classifier gradients. It raises mean PSNR by roughly 7 dB on CIFAR-100 and 8 dB on LFW and lifts the number of near-exact reconstructions ($\text{MSE} < 0.01$) from 3 to 39 of 50 on CIFAR-100; mean SSIM approximately doubles on both datasets (+99% on CIFAR-100, +137% on LFW), confirming that the gain is perceptual as well as numerical (Table 6.2). Retaining part of a layer rather than removing it entirely preserves information that a full mask would lose, while suppressing the remaining entries eases optimization. The full gradient is therefore not the best optimization signal: a smaller retained subset exposes the input more effectively.

7.2.3 Jacobian rank

The Jacobian reaches full rank under both selection modes, so the gradients encode enough independent constraints on the input regardless of which entries are kept; the modes differ only in how quickly they get there, with `topk_abs` needing far more entries than `layer_spread` (46,500 against 7,500 on VGG-11; Figure 6.7). The gap is the FC-dominance effect seen for LeNet, amplified by VGG’s classifier of over 100 million parameters: sitting closest to the loss, its high-magnitude gradients dominate `topk_abs` yet span a low-dimensional subspace that adds little rank, whereas `layer_spread` draws spatially diverse convolutional rows from the start.

7.3 Interpretation of ResNet results

ResNet stands in contrast to VGG: masking generally did not improve its reconstruction, and where improvements appeared at all they were marginal and confined to specific pretrained ResNet-50 configurations. The mechanism responsible for the VGG gains does not transfer to residual architectures, which points to the architecture’s information flow, rather than the optimizer or loss, as the deciding factor. Batch normalization is not the driver either: with a batch size of one it stays neutral during optimization and cannot account for the difference.

7.3.1 Residual connections distribute gradient information

In the non-pretrained setting, no leave-one-layer-out mask improves reconstruction on either network, the inverse of the VGG pattern (Figure 6.6; Appendix C.2.2): every clear effect is a degradation. On ResNet-18, `conv1` is the most damaging layer to mask under L-BFGS, with mean PSNR falling 2.3 dB (17.53 $\rightarrow$ 15.23 dB) and SSIM by 41% (0.54 $\rightarrow$ 0.32); under Signed AdamW its effect is more modest and the early residual groups account for the largest degradation. ResNet-50 shows the same pattern, with `conv1` again the most damaging layer and no mask producing a meaningful improvement (Appendix C.2.2), confirming the result is not specific to basic residual blocks.

This contrast with VGG is consistent with how the two architectures route gradient information. VGG is purely sequential, so layer-wise masking can directly isolate gradient components associated with particular stages of the forward pass. ResNet’s skip connections [12], by re-introducing earlier activations into later blocks, spread input-relevant information across multiple residual paths; removing an

entire layer group is therefore more likely to discard useful constraints along with any poorly conditioned components than to isolate noise. Greater depth compounds the problem: ResNet-18 achieves a non-pretrained baseline of ≈ 17.4 dB, while ResNet-50 reaches only ≈ 13.7 dB (Appendix C.2.2), yet the added depth still produces no exploitable noise structure.

The masking-percentage sweep confirms the same pattern across masking levels (Figure 6.9): unlike VGG-13’s clear peak at 70% masking, ResNet-18’s curve stays flat and trends downward throughout, peaking at only 15 of 100 successful reconstructions near 40% masking and falling toward zero at higher percentages.

Effect of pretraining

Pretraining alters the ResNet masking pattern, though differently from VGG. On ResNet-18 the shift is minimal: `conv1` remains the most damaging layer to mask and becomes more sensitive, with the drop deepening from ≈ 2.3 dB to 3.1 dB under L-BFGS ($17.46 \rightarrow 14.41$ dB) and SSIM falling by 40% ($0.43 \rightarrow 0.26$), and the early residual groups remain broadly harmful. The only deviation is `layer4` under Signed AdamW, where masking yields a marginal improvement ($15.05 \rightarrow 15.43$ dB), a single exception under one optimizer and too isolated to form a clear pattern.

The effect is more pronounced for ResNet-50 (Appendix C.2.2), where pretraining produces a cleaner partial reversal: `conv1`, the dominant degrader in the non-pretrained case, becomes essentially neutral, and several mid-level residual blocks turn from harmful to slightly beneficial to mask, with the improvement appearing across multiple layers under L-BFGS and partially under Signed AdamW. The high per-image variance and limited sample size mean these results cannot support definitive conclusions; they are better read as an indication that masking middle layers may yield improvements for pretrained ResNet-50, one consistent enough across layers not to be disregarded but not conclusive enough to establish as a finding without larger-scale experiments. In both networks, ImageNet weights restructure what the gradients encode away from the low-level structure of random initialization: mid-level layers shift toward high-level semantic features [15] whose gradients reflect feature structure rather than precise pixel values, so removing them can reduce interference rather than discard useful signal. The effect is nonetheless modest overall: pretrained ResNet baselines fall far below any non-pretrained run (Table 2.1), so the gradient signal is fundamentally weaker for pixel-level inversion regardless of masking.

7.3.2 Jacobian rank

In both the non-pretrained and pretrained settings, the structural condition for reconstruction is not the limiting factor. Both ResNet-18 and ResNet-50 achieve full column rank under both selection modes (Table B.1), confirming that the gradients encode sufficient independent constraints on the input in both cases. The failure of masking is therefore better explained by the difficulty of isolating useful constraints through layer-level masking in a residual architecture than by an absence of recoverable information in the retained gradients.

Unlike VGG, the two selection modes perform comparably for ResNet, with full-rank thresholds within a few thousand entries of each other on both networks (Table B.1). The near-equivalence reflects the absence of a large fully connected classifier: ResNet replaces the multi-layer FC stack with a single linear layer after global average pooling, so magnitude-based selection is not pulled toward a dominant low-dimensional subspace. This is consistent with the interpretation that the large gap seen for VGG reflects its classifier architecture rather than a general limitation of `topk_abs`.

7.4 Synthesis

Taken together, the results offer a consistent interpretation of when gradient inversion fails and why masking can help. In sequential architectures such as VGG, particularly under optimization methods that equalize gradient component contributions, the primary limitation is optimization difficulty rather than absence of information. The Jacobian rank analysis (Section 5.1) established this directly: across all tested configurations, the gradient Jacobian achieves full column rank $d = 3,072$, confirming that the gradients retain sufficient independent constraints on the input across the entry pool

configurations tested. The rank reconstruction sweep shows that full rank alone does not guarantee successful reconstruction: when the gradient matching objective is dominated by classifier entries, optimization can fail even after the structural condition is satisfied. On LeNet, both the FC-included and FC-excluded configurations reach full rank, yet PSNR rises to 43.2 dB only once the FC layer is excluded from the matching objective, while the FC-included configuration stays near 5 dB at the same entry count (Figure 6.8). The masking experiments then reveal where the difficulty lies: specific layers, primarily early convolutional layers in the non-pretrained setting, interfere with convergence more than they contribute useful constraints. Removing those components improves reconstruction without removing the information needed to constrain the inverse problem, directly supporting the hypothesis that failures in these settings arise from numerical instability in the optimization. Random restarts corroborate this: because the objective is non-convex (Chapter 4), each restart gives both configurations another chance to escape a poor local minimum, yet on VGG-13 the masked configuration stays ahead at every count, with a gap of approximately 7.34 dB at one restart narrowing slightly to 7.10 dB at five (Figure 6.10). The advantage is therefore a genuinely easier optimization problem, not a fortunate initialization.

ResNet reaches the opposite outcome from the same starting point. Its gradients again attain full column rank, so the failure of masking is not an informational deficit; instead, skip connections spread input-relevant information across many residual paths, and in the non-pretrained setting no leave-one-layer-out mask isolates noise without also removing useful constraints. Restarts reinforce this: even with identical restart opportunities the masked run never overtakes the unmasked baseline (gap $\approx 0.09 \rightarrow 0.22$ dB from $k = 1$ to 5; Figure 6.11), so the absence of benefit is architectural rather than a matter of where optimization happens to start. Whether this is a more fundamental limit on inverting residual architectures or simply calls for a different masking strategy remains a topic for future work.

That architectural difference does not reduce to a question of parameter count. On total parameter count VGG-13 dwarfs ResNet-18, but most of VGG’s parameters lie in its three fully connected classifier layers (≈ 129 M total against ≈ 9 M for the convolutional backbone; Figure E.1), whereas ResNet replaces that stack with a single linear layer after global average pooling. On the convolutional backbone that actually constrains the pixels the two are comparable (≈ 9.4 M and ≈ 11.5 M), in line with their similar non-pretrained baselines. That imbalance is also what makes the classifier worth masking in VGG specifically: its classifier contributes a large block of gradient entries that mostly encode class-level structure, so once iDLG has recovered the label, removing them frees the matching objective to fit the pixel-constraining convolutional gradients and raises non-pretrained reconstruction (Table C.5). ResNet’s single small classifier contributes no comparable block, so masking it offers no similar benefit.

Pretraining changes the picture for both architectures, but not in the same direction. For VGG it makes the early-layer and classifier gradients more informative, so masking them turns from helpful to harmful; for ResNet, the shift is subtler on ResNet-18 (where only `layer4` under Signed AdamW shows a marginal benefit) and more pronounced on ResNet-50, where mid-level blocks turn from harmful to mildly helpful (Appendix C.2.2), though the variance and limited sample size mean this should be read as an indication rather than a definitive finding. The common thread is not a uniform gain or loss but a shift in which gradients carry useful pixel-level information. In both cases the pretrained baselines are also far lower (ResNet-50 reaches only 7.06 dB under L-BFGS), so pretraining leaves a weaker gradient signal overall that masking cannot recover by conditioning alone.

7.5 Limitations and future work

The scope of the experiments was constrained by the computational cost of gradient inversion. Each reconstruction requires repeated forward and backward passes through the target network, and the cost increases further for deeper architectures, longer optimization runs, and additional random restarts. The main ablation studies were therefore conducted on 30 images per configuration with a single restart. This is sufficient to reveal consistent masking patterns, but larger sample sizes would provide more precise estimates and a stronger basis for assessing how reliably the observed effects generalize

across images. The modest sample size is also a likely reason for the frequent rejection of normality in the Shapiro-Wilk tests: with only 30 paired differences per condition, a few outlying images can pull the distribution away from normality, which is why reconstruction quality is summarized with descriptive statistics rather than parametric tests.

The Jacobian rank analysis is limited by the cost of evaluating large numbers of gradient rows. Calculating the Jacobian becomes increasingly expensive as more rows are included, so the analysis is restricted to selected subsets of gradient entries. Reaching full column rank within a subset is sufficient to show that the retained gradients contain enough locally independent constraints on the input, but it does not guarantee successful reconstruction in practice. The reported rank is also a numerical estimate and is therefore sensitive to floating-point precision and the threshold used to distinguish small singular values from numerical noise.

The rank reconstruction sweep should be interpreted as an exploratory analysis rather than definitive evidence. Due to its computational cost, the sweep was conducted on a limited set of configurations and images. While the results suggest that reconstruction quality depends strongly on which gradient entries are retained after full rank is achieved, further experiments are required to determine how these patterns generalize across images, architectures, and initializations. A natural direction for future work is a larger-scale evaluation of the relationship between Jacobian rank, masking strategy, and reconstruction quality.

Several further experimental directions remain open. Total variation regularization was disabled throughout to keep gradient masking as the isolated variable of interest, which makes the masked and unmasked objectives easier to compare but leaves the strongest reconstruction setting from Inverting Gradients unevaluated; since TV regularization suppresses high-frequency artifacts, it could change both the baseline reconstruction quality and the marginal benefit of masking, and masking should therefore be retested across TV weights, especially for the cosine-similarity and Signed AdamW configuration. The experiments were also restricted to a batch size of one, where each gradient corresponds to a single image; larger batches mix the contributions of several samples, and exploring them would test whether masking still isolates useful constraints once gradients are aggregated. Finally, only shallower VGG and ResNet networks were evaluated, so testing deeper or otherwise more complex variants would show whether the same masking trends persist as network capacity grows.

Another possible extension is to combine gradient masking with generative priors. GIDD [25] uses a diffusion model to denoise intermediate reconstructions while still matching the shared gradients. Pairing this with masking is a natural test of the present work: masking would remove the poorly conditioned gradient constraints, while the diffusion prior would steer the remaining optimization toward natural images. Whether the masking patterns identified here remain beneficial under such a learned image prior is left for future work.

Conclusion

This thesis investigated whether unsuccessful gradient inversion necessarily implies that shared gradients lack recoverable information, or whether reconstruction can fail because that information is difficult to extract through optimization. Gradient masking was used as a diagnostic tool for this question. By removing selected gradient components before inversion, the experiments tested whether smaller gradient subsets could provide a more effective reconstruction signal than the full gradient.

The results show that this can occur, but only under specific conditions. The most striking demonstration is VGG-11: masking the classifier together with `features.0` under Signed AdamW produces near-perfect reconstruction, with mean SSIM rising from 0.62 to 0.94 (+52%) and mean MSE falling by 94%. Because VGG-11 left almost no room for further investigation at this level of quality, VGG-13 serves as the primary analysis target: as a deeper network it is harder to invert and closer to architectures used in practice. There, the best masking configuration raises mean PSNR from 15.81 to 23.14 dB on CIFAR-100 and from 14.14 to 22.50 dB on LFW, with mean SSIM improving by 99% and 137% respectively. The layer-wise ablation and masking-percentage experiments support the same interpretation: on VGG-13, successfully reconstructed images ($\text{MSE} < 0.01$) rise from 6 to 40 of 100 at 70% masking before declining, confirming that selected gradients can interfere with optimization, but removing too much of the gradient eventually discards useful reconstruction constraints.

Whether masking helps depends on whether informative and disruptive gradient components can be separated by the network's structure. In sequential networks this is possible: for LeNet, masking early convolutional layers removes the most direct pixel constraints and degrades reconstruction, while masking the fully connected layer improves it by eliminating classifier-level constraints that compete with those convolutional gradients. VGG follows the same logic at greater depth, where disruptive components concentrate in identifiable layers and can be removed without discarding the signal needed for reconstruction. ResNet's skip connections break this separability: input-relevant information is distributed across multiple residual paths, so removing an entire layer group discards useful constraints alongside any poorly conditioned ones, and in the non-pretrained setting no mask improves reconstruction on either network. Pretraining shifts which gradients carry useful pixel-level information across all architectures, showing that the value of a gradient component depends not only on its position in the network but also on the learned representation encoded by the weights.

The numerical rank analysis adds a structural check on this interpretation. Comparatively small retained gradient subsets can still reach full column rank with respect to the input, meaning that masking does not necessarily remove the local information required for reconstruction. However, full rank alone does not guarantee high-quality reconstruction: even when the gradients contain sufficient independent constraints on the input, a poorly conditioned matching objective can prevent the optimizer from finding them. The restart experiments reinforce this point. Additional initializations improve both masked and unmasked reconstruction, but on VGG-13 the masked configuration stays ahead at every restart count, indicating that the benefits of masking are not due to initialization.

The research question can therefore be answered with an important qualification. Poor reconstruction quality does not, by itself, demonstrate that gradients are safe or uninformative. In some settings, the relevant information is present but obscured by gradient components that make the inversion objective harder to optimize, and selective masking can expose this hidden vulnerability. At the same time, masking is not a universal improvement: its effectiveness depends on the architecture, weight

initialization, and reconstruction method. These findings suggest that privacy risk in gradient sharing cannot be assessed only from the success or failure of a baseline inversion attack, but must also consider whether the informative parts of the gradient can be isolated and exploited.

Bibliography

- [1] H. Brendan McMahan et al. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. 2017. arXiv: 1602.05629v4 [cs.LG]. url: <https://arxiv.org/abs/1602.05629v4>.
- [2] Ligeng Zhu, Zhijian Liu, and Song Han. *Deep Leakage from Gradients*. 2019. arXiv: 1906.08935 [cs.LG]. url: <https://arxiv.org/abs/1906.08935>.
- [3] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. *iDLG: Improved Deep Leakage from Gradients*. 2020. arXiv: 2001.02610 [cs.LG]. url: <https://arxiv.org/abs/2001.02610>.
- [4] Jonas Geiping et al. *Inverting Gradients – How easy is it to break privacy in federated learning?* 2020. arXiv: 2003.14053 [cs.CV]. url: <https://arxiv.org/abs/2003.14053>.
- [5] Jia Qian, Hiba Nassar, and Lars Kai Hansen. *Minimal Model Structure Analysis for Input Reconstruction in Federated Learning*. arXiv:2010.15718 [cs.CR]. 2020. arXiv: 2010.15718 [cs.CR].
- [6] *Illustration of CIFAR-100 Dataset Examples*. 2022. url: https://www.researchgate.net/figure/Illustration-of-CIFAR-100-dataset-examples-CIFAR-100-contains-tiny-images-of-100-classes_fig1_364953611 (visited on 05/27/2026).
- [7] Jessica Li. *LFW Dataset*. 2018. url: <https://www.kaggle.com/datasets/jessicali9530/lfw-dataset> (visited on 05/27/2026).
- [8] Gary B. Huang et al. “Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments”. In: *Workshop on Faces in ‘Real-Life’ Images: Detection, Alignment, and Recognition*. Erik Learned-Miller and Andras Ferencz and Frédéric Jurie. Marseille, France, Oct. 2008. url: <https://inria.hal.science/inria-00321923>.
- [9] Jessica Li. *LFW Dataset*. 2019. url: <https://www.kaggle.com/datasets/jessicali9530/lfw-dataset> (visited on 05/04/2025).
- [10] Jorge Cardete. *Convolutional Neural Networks: A Comprehensive Guide*. <https://medium.com/thedeephub/convolutional-neural-networks-a-comprehensive-guide-5cc0b5eae175>. Accessed: 2026-06-04. Feb. 2024.
- [11] Y. Lecun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. doi: 10.1109/5.726791.
- [12] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
- [13] Jiayao Wang et al. “Optical Ship Target Detection Method Combining Hierarchical Search and Visual Residual Network”. In: *Opto-Electronic Engineering* 48.4 (2021), p. 200249. doi: 10.12086/oe.2021.200249. url: <https://www.ojournal.org/oe/en/article/doi/10.12086/oe.2021.200249>.
- [14] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *arXiv preprint arXiv:1409.1556* (2015).
- [15] Matthew D. Zeiler and Rob Fergus. “Visualizing and Understanding Convolutional Networks”. In: *Computer Vision – ECCV 2014*. Springer International Publishing, 2014, pp. 818–833.
- [16] Peter Beelen. *Mat1a Notes*. 2023.
- [17] Gayatri Nikam. *Convex vs Non-Convex Functions in ML: Navigating the Peaks and Valleys*. Sept. 2024. url: <https://medium.com/@gayatrinikam5103/convex-vs-non-convex-functions-in-ml-navigating-the-peaks-and-valleys-011e25409c6e> (visited on 05/26/2026).
- [18] Dong C. Liu and Jorge Nocedal. “On the limited memory BFGS method for large scale optimization”. In: *Mathematical Programming* 45 (1989), pp. 503–528. issn: 1436-4646. doi: 10.1007/BF01589116. url: <https://doi.org/10.1007/BF01589116>.
- [19] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. url: <https://arxiv.org/abs/1412.6980>.

- [20] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: *7th International Conference on Learning Representations (ICLR)*. 2019. url: <https://arxiv.org/abs/1711.05101>.
- [21] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. “Deep Sparse Rectifier Neural Networks”. In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011.
- [22] Zhou Wang and Alan C. Bovik. “Mean Squared Error: Love It or Leave It? A New Look at Signal Fidelity Measures”. In: *IEEE Signal Processing Magazine* 26.1 (2009), pp. 98–117. doi: 10.1109/MSP.2008.930649. url: <https://doi.org/10.1109/MSP.2008.930649>.
- [23] Zhou Wang et al. “Image Quality Assessment: From Error Visibility to Structural Similarity”. In: *IEEE Transactions on Image Processing* 13.4 (2004), pp. 600–612. doi: 10.1109/TIP.2003.819861. url: <https://doi.org/10.1109/TIP.2003.819861>.
- [24] S. S. Shapiro and M. B. Wilk. “An analysis of variance test for normality (complete samples)”. In: *Biometrika* 52.3-4 (Dec. 1965), pp. 591–611. issn: 0006-3444. doi: 10.1093/biomet/52.3-4.591. eprint: <https://academic.oup.com/biomet/article-pdf/52/3-4/591/962907/52-3-4-591.pdf>. url: <https://doi.org/10.1093/biomet/52.3-4.591>.
- [25] Xuebo Wang, Hongguang Sun, and Yi He. “GIDD: Gradient inversion using diffusion model for denoising in federated learning”. In: *Neurocomputing* 672 (2026), p. 132682. issn: 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2026.132682>. url: <https://www.sciencedirect.com/science/article/pii/S0925231226000792>.

Code documentation

The code and its history is sourced in our GitHub repository accessible through: <https://github.com/MathiasKES/stable-ginv>.

Furthermore, the code has been automatically documented using <https://www.sphinx-doc.org> at: <https://mathiaskes.github.io/stable-ginv>

Jacobian Rank

Table B.1. Minimum number of gradient entries required to reach full Jacobian rank ($d = 3,072$) on CIFAR-100, evaluated per architecture and entry selection mode. Each entry reports the smallest row count at which the Jacobian first achieved full column rank, averaged over 30 samples. The FC layer is included in the selection pool and competes with convolutional entries by magnitude.

Select mode	Network	Unknowns	Gradients to full rank
layer_spread	LeNet	3072	6,000
	VGG-11	3072	7,500
	VGG-13	3072	6,500
	ResNet-18	3072	9,500
	ResNet-50	3072	9,500
topk_abs	LeNet	3072	6,500
	VGG-11	3072	46,500
	VGG-13	3072	30,000
	ResNet-18	3072	6,500
	ResNet-50	3072	10,000

B.1 Rank Reconstruction Sweep

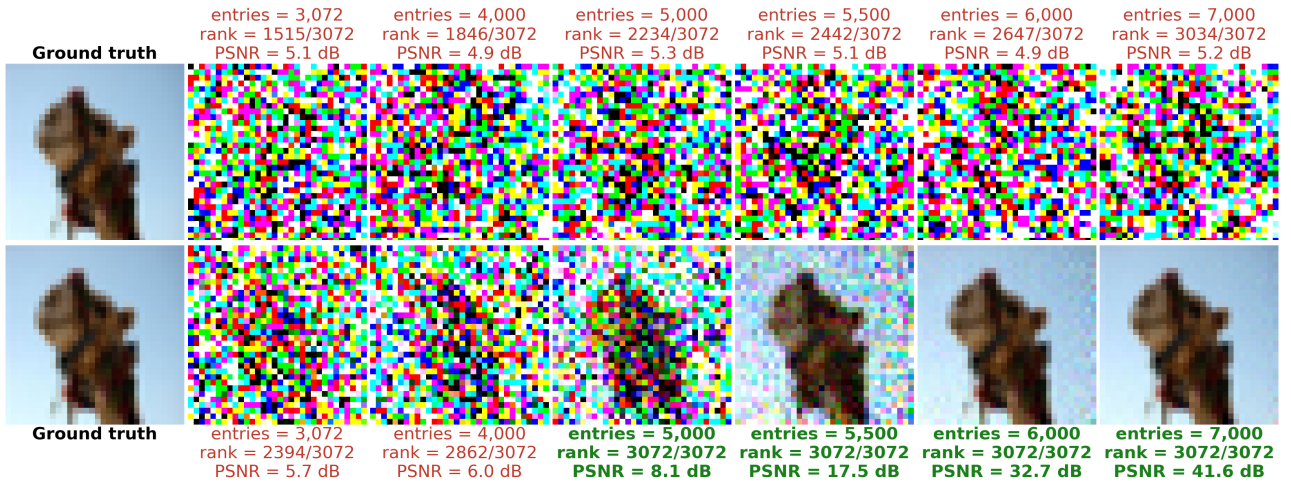


Figure B.1. Rank reconstruction sweep using `topk_abs` entry selection on LeNet / CIFAR-100. Top: FC layer included in the entry pool; bottom: FC layer excluded. Labels above each top panel and below each bottom panel show retained entries, Jacobian rank, and PSNR. Green (bold) = full rank (rank = $d = 3,072$); red = rank deficit. With FC included, full rank is never achieved within the tested range (reaching at most 3,034/3,072 at 7,000 entries); with FC excluded, full rank is reached at 5,000 entries and PSNR rises to 41.6 dB at 7,000 entries.

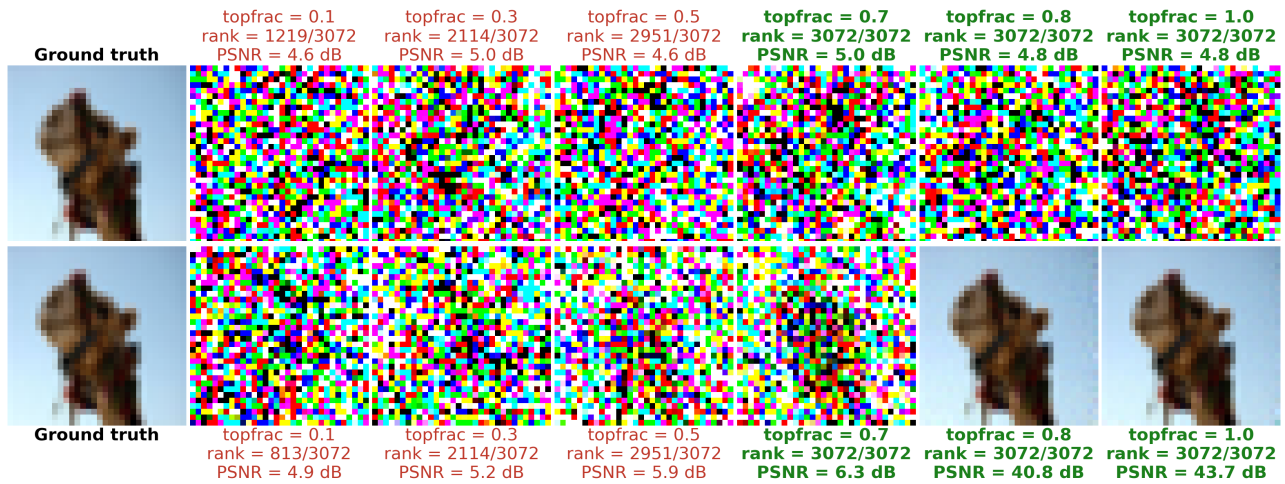


Figure B.2. Rank reconstruction sweep using `gradsize_topfrac_entries_layer` entry selection on LeNet / CIFAR-100. Top: FC layer included; bottom: FC layer excluded. Same label convention as Figure 6.8. Both conditions reach full rank at the same threshold ($\text{topfrac} = 0.7$); with FC excluded, PSNR reaches 40.8 dB at $\text{topfrac} = 0.8$ and 43.7 dB at $\text{topfrac} = 1.0$, while the FC-included condition does not exceed 5.0 dB. This selection mode matches the masking-percentage sweep; see Figure 6.9.

Ablation studies

C.1 LeNet

Masked layer	MSE	PSNR (dB)	SSIM	SW p_{MSE}	SW p_{PSNR}	SW p_{SSIM}
L-BFGS / No pretrain (300 iterations)						
None	0.04 ± 0.09	40.98 ± 15.00	0.87 ± 0.33	–	–	–
body.0	0.08 ± 0.11	13.58 ± 4.31	0.41 ± 0.23	< 0.001	< 0.001	0.03
body.2	0.26 ± 0.03	5.84 ± 0.43	0.03 ± 0.01	< 0.001	< 0.001	< 0.001
body.4	0.05 ± 0.10	37.88 ± 16.95	0.80 ± 0.39	< 0.001	< 0.001	< 0.001
fc	0.01 ± 0.03	45.49 ± 8.99	0.97 ± 0.15	< 0.001	< 0.001	< 0.001
Signed AdamW / No pretrain (5000 iterations)						
None	0.11 ± 0.04	9.78 ± 1.85	0.13 ± 0.07	–	–	–
body.0	0.13 ± 0.04	8.96 ± 1.31	0.10 ± 0.05	0.16	0.00	0.00
body.2	0.15 ± 0.03	8.46 ± 1.05	0.08 ± 0.04	0.14	0.01	0.01
body.4	0.11 ± 0.04	9.89 ± 1.68	0.14 ± 0.08	0.00	0.07	< 0.001
fc	0.09 ± 0.02	10.85 ± 1.26	0.20 ± 0.08	0.03	0.81	0.89

Table C.1. Summary of LeNet reconstruction experiments on CIFAR-100 over 30 runs. The masked layer indicates the layer whose gradients were removed before reconstruction. Lower MSE and higher PSNR and SSIM correspond to better reconstruction quality. SW denotes the p-value of the Shapiro-Wilk normality test applied to the paired differences between each masking configuration and the corresponding unmasked baseline. Red numbers indicate rejection of null hypothesis at a $\alpha = 0.05$ significance level.

C.2 ResNet

C.2.1 ResNet-18

Masked layer	MSE	PSNR (dB)	SSIM	SW p_{MSE}	SW p_{PSNR}	SW p_{SSIM}
L-BFGS / No pretrain						
None	0.02 ± 0.01	17.53 ± 2.41	0.54 ± 0.16	–	–	–
conv1	0.03 ± 0.01	15.23 ± 1.99	0.32 ± 0.09	0.80	0.33	0.10
bn1	0.02 ± 0.01	17.68 ± 2.65	0.54 ± 0.15	0.41	0.66	0.37
layer1	0.03 ± 0.02	16.31 ± 2.34	0.46 ± 0.14	0.01	1.00	0.07
layer2	0.03 ± 0.02	16.50 ± 2.74	0.47 ± 0.17	< 0.001	0.29	0.38
layer3	0.02 ± 0.02	17.08 ± 2.83	0.50 ± 0.18	< 0.001	0.68	0.18
layer4	0.02 ± 0.02	17.55 ± 2.68	0.55 ± 0.16	< 0.001	0.99	0.80
fc	0.02 ± 0.01	17.50 ± 2.47	0.54 ± 0.13	0.07	0.66	0.55
L-BFGS / Pretrained						
None	0.02 ± 0.02	17.46 ± 3.42	0.43 ± 0.18	–	–	–
conv1	0.04 ± 0.02	14.41 ± 1.77	0.26 ± 0.10	< 0.001	0.42	0.35
bn1	0.02 ± 0.02	17.54 ± 3.58	0.44 ± 0.19	0.08	0.27	0.43
layer1	0.02 ± 0.02	17.57 ± 3.64	0.47 ± 0.17	0.01	0.79	0.19
layer2	0.03 ± 0.02	17.02 ± 3.54	0.39 ± 0.19	0.07	0.78	0.22
layer3	0.04 ± 0.03	16.16 ± 4.41	0.34 ± 0.25	0.03	0.34	0.31
layer4	0.05 ± 0.04	14.58 ± 4.38	0.26 ± 0.22	0.002	0.91	0.27
fc	0.03 ± 0.02	16.56 ± 3.72	0.38 ± 0.19	0.004	0.66	0.81
Signed AdamW / No pretrain						
None	0.02 ± 0.01	17.13 ± 2.61	0.52 ± 0.16	–	–	–
conv1	0.02 ± 0.01	16.79 ± 2.54	0.43 ± 0.11	0.20	0.16	0.92
bn1	0.02 ± 0.02	17.27 ± 2.59	0.53 ± 0.16	0.06	0.97	0.12
layer1	0.03 ± 0.02	15.87 ± 2.24	0.44 ± 0.14	0.04	0.68	0.17
layer2	0.03 ± 0.02	16.30 ± 2.61	0.47 ± 0.16	< 0.001	0.43	0.11
layer3	0.03 ± 0.02	16.63 ± 2.43	0.49 ± 0.16	< 0.001	0.18	0.76
layer4	0.02 ± 0.01	16.98 ± 2.42	0.52 ± 0.16	0.32	0.86	0.43
fc	0.03 ± 0.02	16.65 ± 2.43	0.49 ± 0.15	< 0.001	0.001	0.56
Signed AdamW / Pretrained						
None	0.03 ± 0.01	15.05 ± 1.61	0.33 ± 0.11	–	–	–
conv1	0.06 ± 0.01	12.31 ± 0.92	0.21 ± 0.08	0.15	0.74	0.45
bn1	0.04 ± 0.02	14.78 ± 1.83	0.31 ± 0.12	< 0.001	0.001	0.01
layer1	0.04 ± 0.02	14.23 ± 1.70	0.28 ± 0.09	0.89	0.05	0.14
layer2	0.04 ± 0.02	14.36 ± 1.85	0.25 ± 0.08	< 0.001	0.05	0.01
layer3	0.04 ± 0.02	14.64 ± 1.89	0.30 ± 0.10	0.86	0.91	0.001
layer4	0.03 ± 0.02	15.43 ± 2.52	0.35 ± 0.11	< 0.001	0.21	0.09
fc	0.03 ± 0.01	15.45 ± 1.91	0.36 ± 0.12	0.003	0.14	0.43

Table C.2. Summary of ResNet-18 reconstruction experiments on CIFAR-100 over 30 runs.

C.2.2 ResNet-50

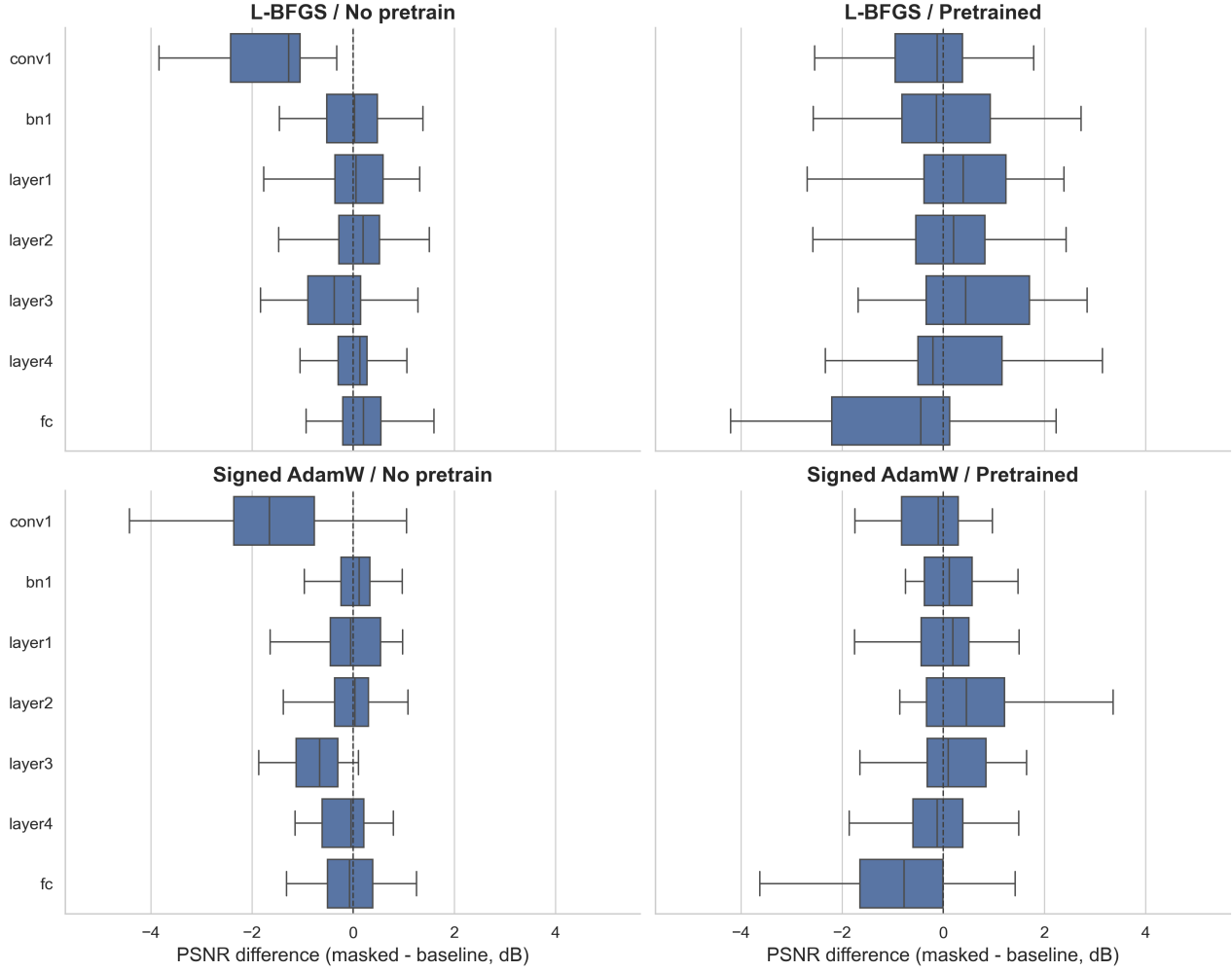


Figure C.1. Layer-wise masking ablation for ResNet-50 on CIFAR-100 across optimizer/loss and pretraining settings. Each box plot shows the per-sample PSNR difference between the masked reconstruction and the unmasked baseline; the dashed vertical line marks no change from the baseline. Negative values indicate lower reconstruction quality after masking.

The ablation on ResNet-50 shows that **conv1** is the most damaging layer to mask in the non-pretrained setting, with mean PSNR falling from 13.69 dB to 12.00 dB under L-BFGS and from 13.67 dB to 12.00 dB under Signed AdamW. The remaining layer groups remain close to baseline in both non-pretrained configurations. With pretrained weights, the pattern differs: **conv1** has little effect under L-BFGS (7.06 $\rightarrow$ 7.05 dB), and masking middle layers (**layer1**–**layer3**) yields slight improvements above the 7.06 dB baseline (7.64, 7.57, and 7.73 dB respectively). Under pretrained Signed AdamW, **layer2** similarly improves above baseline (9.50 $\rightarrow$ 10.05 dB). The **fc** layer degrades reconstruction in the pretrained setting under both optimizers (7.06 $\rightarrow$ 6.26 dB under L-BFGS and 9.50 $\rightarrow$ 8.56 dB under Signed AdamW), indicating that the classifier gradient adds noise to the objective once pretrained representations are used. Per-layer statistics are given in Table C.3.

Masked layer	MSE	PSNR (dB)	SSIM	SW p_{MSE}	SW p_{PSNR}	SW p_{SSIM}
L-BFGS / No pretrain						
None	0.05 ± 0.03	13.69 ± 2.62	0.20 ± 0.11	–	–	–
conv1	0.07 ± 0.03	12.00 ± 1.91	0.10 ± 0.05	0.05	0.01	0.29
bn1	0.05 ± 0.03	13.67 ± 2.52	0.22 ± 0.11	0.87	0.57	0.81
layer1	0.05 ± 0.03	13.87 ± 2.65	0.22 ± 0.11	0.07	0.45	0.23
layer2	0.05 ± 0.02	13.83 ± 2.68	0.20 ± 0.10	0.22	0.79	0.20
layer3	0.05 ± 0.03	13.23 ± 2.28	0.16 ± 0.09	0.99	0.37	0.89
layer4	0.05 ± 0.03	13.79 ± 2.67	0.20 ± 0.12	0.01	< 0.001	0.12
fc	0.05 ± 0.03	13.96 ± 2.66	0.23 ± 0.11	< 0.001	0.06	0.11
L-BFGS / Pretrained						
None	0.20 ± 0.06	7.06 ± 1.23	0.01 ± 0.02	–	–	–
conv1	0.21 ± 0.07	7.05 ± 1.47	0.00 ± 0.01	0.01	0.00	0.87
bn1	0.21 ± 0.08	7.16 ± 1.66	0.00 ± 0.02	0.10	0.01	0.01
layer1	0.19 ± 0.08	7.64 ± 1.69	0.01 ± 0.02	0.34	0.16	0.01
layer2	0.18 ± 0.06	7.57 ± 1.47	0.01 ± 0.01	0.03	0.04	0.63
layer3	0.18 ± 0.05	7.73 ± 1.19	0.00 ± 0.02	0.10	0.12	0.01
layer4	0.19 ± 0.07	7.43 ± 1.58	0.00 ± 0.01	0.11	0.03	0.69
fc	0.25 ± 0.07	6.26 ± 1.43	0.00 ± 0.02	0.23	0.51	0.70
Signed AdamW / No pretrain						
None	0.05 ± 0.03	13.67 ± 2.51	0.21 ± 0.11	–	–	–
conv1	0.07 ± 0.03	12.00 ± 2.04	0.12 ± 0.05	0.82	0.98	0.83
bn1	0.05 ± 0.03	13.70 ± 2.59	0.22 ± 0.11	< 0.001	0.69	0.25
layer1	0.05 ± 0.03	13.62 ± 2.61	0.21 ± 0.11	0.02	0.23	0.77
layer2	0.05 ± 0.03	13.65 ± 2.54	0.20 ± 0.09	0.05	0.27	0.84
layer3	0.06 ± 0.03	12.90 ± 2.44	0.15 ± 0.10	0.01	0.23	0.29
layer4	0.05 ± 0.03	13.38 ± 2.61	0.20 ± 0.11	0.00	< 0.001	< 0.001
fc	0.05 ± 0.03	13.69 ± 2.53	0.23 ± 0.11	0.87	0.87	0.78
Signed AdamW / Pretrained						
None	0.12 ± 0.04	9.50 ± 1.67	0.04 ± 0.07	–	–	–
conv1	0.12 ± 0.04	9.27 ± 1.38	0.04 ± 0.05	0.23	0.41	0.01
bn1	0.11 ± 0.04	9.72 ± 1.75	0.05 ± 0.07	0.15	0.06	0.73
layer1	0.12 ± 0.04	9.50 ± 1.53	0.05 ± 0.05	0.33	0.58	0.49
layer2	0.11 ± 0.04	10.05 ± 1.67	0.05 ± 0.05	0.00	0.02	0.14
layer3	0.12 ± 0.04	9.60 ± 1.62	0.04 ± 0.05	0.55	0.16	0.03
layer4	0.12 ± 0.04	9.44 ± 1.82	0.04 ± 0.07	0.96	0.84	0.01
fc	0.14 ± 0.04	8.56 ± 1.14	0.03 ± 0.04	0.13	0.05	< 0.001

Table C.3. Summary of ResNet-50 reconstruction experiments on CIFAR-100 over 30 runs. The masked layer indicates the layer whose gradients were removed before reconstruction. Lower MSE and higher PSNR and SSIM correspond to better reconstruction quality. SW denotes the p-value of the Shapiro-Wilk normality test applied to the paired differences between each masking configuration and the corresponding unmasked baseline. Red numbers indicate rejection of null hypothesis at a $\alpha = 0.05$ significance level.

C.3 VGG

C.3.1 VGG-11

Masked layer	MSE	PSNR (dB)	SSIM	SW p_{MSE}	SW p_{PSNR}	SW p_{SSIM}
L-BFGS / No pretrain						
None	0.05 ± 0.03	13.86 ± 2.85	0.19 ± 0.12	–	–	–
features.0	0.02 ± 0.01	20.40 ± 5.18	0.57 ± 0.22	< 0.001	0.01	0.14
features.3	0.01 ± 0.01	20.64 ± 3.20	0.65 ± 0.12	0.01	0.59	0.41
features.6	0.01 ± 0.01	20.69 ± 3.27	0.66 ± 0.14	0.04	0.46	0.24
features.8	0.02 ± 0.01	19.66 ± 4.12	0.59 ± 0.18	0.32	0.01	0.28
features.11	0.01 ± 0.01	21.32 ± 3.31	0.70 ± 0.13	0.11	0.53	0.75
features.13	0.01 ± 0.01	21.86 ± 3.79	0.72 ± 0.14	0.01	0.38	0.31
features.16	0.01 ± 0.01	22.53 ± 3.02	0.77 ± 0.11	0.04	0.66	0.02
classifier	0.01 ± 0.01	21.76 ± 3.86	0.70 ± 0.15	0.05	0.03	0.36
L-BFGS / Pretrained						
None	0.05 ± 0.03	14.44 ± 3.63	0.26 ± 0.23	–	–	–
features.0	0.15 ± 0.08	8.70 ± 2.26	0.01 ± 0.02	0.01	0.13	0.01
features.3	0.15 ± 0.07	8.50 ± 1.80	0.01 ± 0.02	0.03	0.17	0.01
features.6	0.15 ± 0.09	8.73 ± 2.21	0.02 ± 0.02	0.01	0.56	0.00
features.8	0.14 ± 0.07	8.90 ± 2.11	0.02 ± 0.02	0.00	0.06	0.00
features.11	0.15 ± 0.08	8.67 ± 2.10	0.02 ± 0.03	0.02	0.22	0.00
features.13	0.14 ± 0.07	9.10 ± 2.36	0.02 ± 0.03	0.03	0.45	0.00
features.16	0.16 ± 0.08	8.40 ± 2.17	0.02 ± 0.03	0.29	0.24	0.00
classifier	0.16 ± 0.08	8.55 ± 2.03	0.01 ± 0.02	0.01	0.03	0.00
Signed AdamW / No pretrain						
None	0.02 ± 0.02	20.57 ± 4.96	0.62 ± 0.24	–	–	–
features.0	0.00 ± 0.00	31.44 ± 3.19	0.94 ± 0.05	< 0.001	0.09	0.00
features.3	0.01 ± 0.00	22.51 ± 2.63	0.75 ± 0.09	< 0.001	0.13	< 0.001
features.6	0.01 ± 0.01	21.93 ± 2.74	0.72 ± 0.10	< 0.001	0.01	< 0.001
features.8	0.01 ± 0.01	21.74 ± 3.14	0.72 ± 0.12	< 0.001	0.02	< 0.001
features.11	0.01 ± 0.01	22.69 ± 3.01	0.76 ± 0.11	< 0.001	0.00	< 0.001
features.13	0.01 ± 0.01	23.03 ± 3.18	0.77 ± 0.11	< 0.001	0.00	< 0.001
features.16	0.01 ± 0.00	23.64 ± 2.92	0.78 ± 0.10	< 0.001	0.01	< 0.001
classifier	0.00 ± 0.00	24.20 ± 3.03	0.80 ± 0.10	< 0.001	0.01	< 0.001
Signed AdamW / Pretrained						
None	0.08 ± 0.03	11.50 ± 1.53	0.13 ± 0.08	–	–	–
features.0	0.14 ± 0.03	8.69 ± 0.91	0.06 ± 0.04	0.69	0.75	0.28
features.3	0.13 ± 0.03	8.87 ± 1.08	0.06 ± 0.05	0.66	0.77	0.43
features.6	0.14 ± 0.03	8.78 ± 0.97	0.06 ± 0.04	0.03	0.19	0.82
features.8	0.14 ± 0.03	8.79 ± 0.98	0.07 ± 0.05	0.17	0.36	0.60
features.11	0.13 ± 0.03	8.97 ± 0.95	0.06 ± 0.05	0.51	0.37	0.13
features.13	0.13 ± 0.03	8.97 ± 1.05	0.06 ± 0.06	0.20	0.73	0.24
features.16	0.13 ± 0.03	8.82 ± 0.98	0.06 ± 0.04	0.59	0.93	0.88
classifier	0.13 ± 0.03	8.87 ± 1.11	0.06 ± 0.07	0.95	0.06	0.06

Table C.4. Summary of VGG-11 reconstruction experiments on CIFAR-100 over 30 runs. The classifier gradients are fully masked in all masked VGG-11 conditions; feature rows indicate the additional feature layer removed before reconstruction. Values report mean $\pm$ standard deviation across per-sample reconstruction metrics. SW denotes the Shapiro-Wilk normality-test p-value for paired differences relative to the corresponding unmasked baseline. Red values indicate $p < 0.05$.

C.3.2 VGG-13

Masked layer	MSE	PSNR (dB)	SSIM	SW p_{MSE}	SW p_{PSNR}	SW p_{SSIM}
L-BFGS / No pretrain						
None	0.06 $\pm$ 0.03	12.61 $\pm$ 2.15	0.12 $\pm$ 0.07	–	–	–
features.0	0.03 $\pm$ 0.02	15.89 $\pm$ 3.52	0.34 $\pm$ 0.19	0.00	< 0.001	< 0.001
features.2	0.03 $\pm$ 0.02	16.34 $\pm$ 3.10	0.39 $\pm$ 0.16	0.00	0.06	0.18
features.5	0.03 $\pm$ 0.01	16.71 $\pm$ 2.91	0.41 $\pm$ 0.16	0.00	0.22	0.47
features.7	0.03 $\pm$ 0.02	16.14 $\pm$ 2.68	0.38 $\pm$ 0.14	0.00	0.37	0.72
features.10	0.03 $\pm$ 0.01	16.12 $\pm$ 2.68	0.39 $\pm$ 0.14	0.00	0.11	0.64
features.12	0.03 $\pm$ 0.02	15.67 $\pm$ 3.06	0.34 $\pm$ 0.16	0.00	0.02	0.01
features.15	0.03 $\pm$ 0.02	15.92 $\pm$ 2.99	0.37 $\pm$ 0.16	0.03	0.01	0.31
features.17	0.03 $\pm$ 0.02	16.40 $\pm$ 2.98	0.41 $\pm$ 0.17	0.04	0.06	0.19
features.20	0.02 $\pm$ 0.01	17.10 $\pm$ 3.10	0.46 $\pm$ 0.16	0.01	0.06	0.51
features.22	0.02 $\pm$ 0.01	17.60 $\pm$ 3.13	0.50 $\pm$ 0.15	0.01	0.55	0.55
classifier	0.03 $\pm$ 0.01	16.73 $\pm$ 3.02	0.43 $\pm$ 0.15	0.01	0.17	0.36
L-BFGS / Pretrained						
None	0.07 $\pm$ 0.04	12.35 $\pm$ 2.47	0.12 $\pm$ 0.13	–	–	–
features.0	0.14 $\pm$ 0.08	9.19 $\pm$ 2.06	0.02 $\pm$ 0.01	< 0.001	0.00	< 0.001
features.2	0.14 $\pm$ 0.07	9.06 $\pm$ 1.75	0.02 $\pm$ 0.02	< 0.001	0.03	< 0.001
features.5	0.14 $\pm$ 0.07	8.80 $\pm$ 1.76	0.02 $\pm$ 0.01	< 0.001	0.04	< 0.001
features.7	0.13 $\pm$ 0.05	9.27 $\pm$ 1.65	0.02 $\pm$ 0.01	< 0.001	< 0.001	< 0.001
features.10	0.12 $\pm$ 0.06	9.56 $\pm$ 1.70	0.02 $\pm$ 0.01	< 0.001	0.01	< 0.001
features.12	0.12 $\pm$ 0.05	9.69 $\pm$ 1.62	0.02 $\pm$ 0.01	< 0.001	0.01	< 0.001
features.15	0.12 $\pm$ 0.06	9.67 $\pm$ 1.93	0.02 $\pm$ 0.01	< 0.001	< 0.001	< 0.001
features.17	0.13 $\pm$ 0.07	9.37 $\pm$ 2.08	0.02 $\pm$ 0.01	< 0.001	0.00	< 0.001
features.20	0.14 $\pm$ 0.07	9.24 $\pm$ 2.14	0.02 $\pm$ 0.01	< 0.001	0.02	< 0.001
features.22	0.17 $\pm$ 0.09	8.38 $\pm$ 2.28	0.01 $\pm$ 0.02	< 0.001	0.06	< 0.001
classifier	0.13 $\pm$ 0.08	9.29 $\pm$ 2.02	0.01 $\pm$ 0.01	< 0.001	0.02	< 0.001
Signed AdamW / No pretrain						
None	0.03 $\pm$ 0.03	15.81 $\pm$ 3.01	0.38 $\pm$ 0.15	–	–	–
features.0	0.01 $\pm$ 0.01	21.26 $\pm$ 4.45	0.68 $\pm$ 0.17	< 0.001	0.07	0.11
features.2	0.02 $\pm$ 0.01	19.07 $\pm$ 3.01	0.60 $\pm$ 0.13	< 0.001	0.18	0.67
features.5	0.02 $\pm$ 0.01	18.39 $\pm$ 2.87	0.56 $\pm$ 0.12	< 0.001	0.82	0.75
features.7	0.02 $\pm$ 0.01	17.58 $\pm$ 2.81	0.51 $\pm$ 0.12	< 0.001	0.77	0.95
features.10	0.02 $\pm$ 0.01	17.26 $\pm$ 2.86	0.48 $\pm$ 0.13	< 0.001	0.37	0.11
features.12	0.02 $\pm$ 0.01	17.03 $\pm$ 2.87	0.47 $\pm$ 0.13	< 0.001	0.92	0.38
features.15	0.02 $\pm$ 0.01	17.43 $\pm$ 2.93	0.49 $\pm$ 0.14	< 0.001	0.44	0.13
features.17	0.02 $\pm$ 0.01	17.69 $\pm$ 2.85	0.52 $\pm$ 0.13	< 0.001	0.12	0.56
features.20	0.02 $\pm$ 0.01	18.12 $\pm$ 3.02	0.54 $\pm$ 0.14	< 0.001	0.53	0.65
features.22	0.02 $\pm$ 0.01	18.64 $\pm$ 3.07	0.57 $\pm$ 0.13	< 0.001	0.68	0.57
classifier	0.02 $\pm$ 0.01	18.57 $\pm$ 3.06	0.57 $\pm$ 0.14	< 0.001	0.23	0.21
Signed AdamW / Pretrained						
None	0.09 $\pm$ 0.03	10.41 $\pm$ 1.14	0.10 $\pm$ 0.05	–	–	–
features.0	0.15 $\pm$ 0.03	8.21 $\pm$ 0.90	0.04 $\pm$ 0.03	0.64	0.85	0.11
features.2	0.15 $\pm$ 0.03	8.34 $\pm$ 0.91	0.05 $\pm$ 0.04	0.51	0.09	0.51
features.5	0.15 $\pm$ 0.04	8.27 $\pm$ 0.93	0.05 $\pm$ 0.05	0.82	0.98	0.20
features.7	0.16 $\pm$ 0.03	8.15 $\pm$ 0.83	0.03 $\pm$ 0.04	0.25	0.53	0.36
features.10	0.15 $\pm$ 0.04	8.32 $\pm$ 0.93	0.05 $\pm$ 0.04	0.80	0.98	0.69
features.12	0.14 $\pm$ 0.03	8.49 $\pm$ 0.91	0.05 $\pm$ 0.04	0.33	0.78	0.33
features.15	0.15 $\pm$ 0.03	8.23 $\pm$ 0.86	0.04 $\pm$ 0.03	0.01	0.80	0.13
features.17	0.15 $\pm$ 0.03	8.38 $\pm$ 0.89	0.04 $\pm$ 0.04	0.28	0.34	0.81
features.20	0.15 $\pm$ 0.03	8.41 $\pm$ 0.95	0.04 $\pm$ 0.06	0.21	0.28	0.13
features.22	0.14 $\pm$ 0.03	8.53 $\pm$ 1.00	0.05 $\pm$ 0.04	0.77	0.78	0.39
classifier	0.15 $\pm$ 0.03	8.40 $\pm$ 0.87	0.04 $\pm$ 0.05	0.65	0.34	0.02

Table C.5. Summary of VGG-13 reconstruction experiments on CIFAR-100 over 30 runs. The classifier gradients are fully masked in all masked VGG-13 conditions; feature rows indicate the additional feature layer removed before reconstruction. Values report mean $\pm$ standard deviation across per-sample reconstruction metrics. SW denotes the Shapiro-Wilk normality-test p-value for paired differences relative to the corresponding unmasked baseline. Red values indicate $p < 0.05$.

Additional pictures

D.1 VGG-11

Signed AdamW, cosine-similarity loss, 5000 iterations

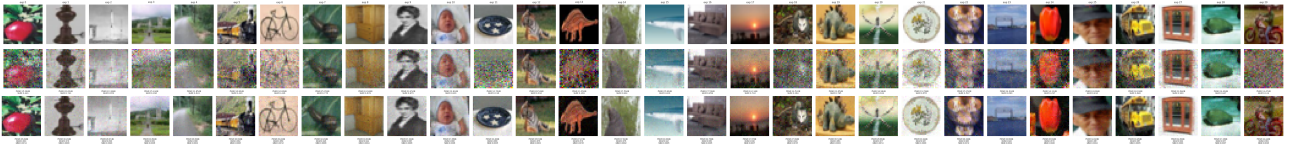


Figure D.1. Gradient inversion reconstructions for CIFAR-100 samples 1-30 using VGG-11 masking classifier and `features.0`. The top row shows the ground-truth images, the second row shows reconstructions obtained from the full gradient, and the third row shows reconstructions obtained after masking the `classifier` and `features.0`. All experiments use Signed AdamW optimization with cosine-similarity loss for 5000 iterations.

L-BFGS, L2 loss, 300 iterations

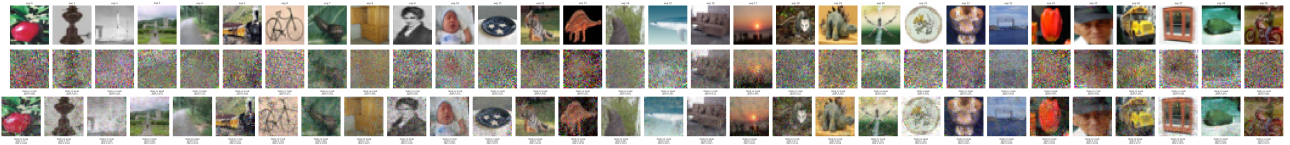
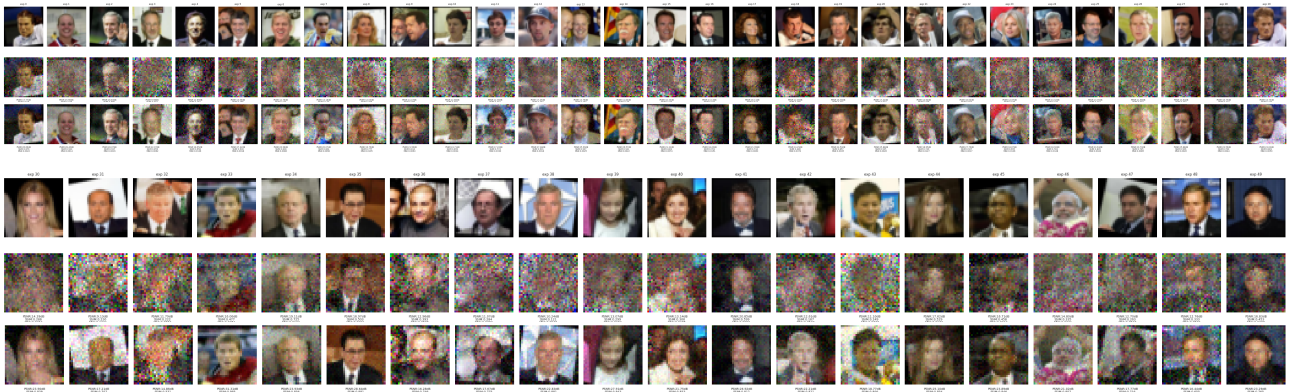


Figure D.2. Gradient inversion reconstructions for CIFAR-100 samples 1-30 using VGG-11 masking classifier and `features.16`. The top row shows the ground-truth images, the second row shows reconstructions obtained from the full gradient, and the third row shows reconstructions obtained after masking the `classifier` and `features.16`. All experiments use L-BFGS optimization with L2 loss for 300 iterations.

D.2 VGG-13 best masking config

LFW



CIFAR-100

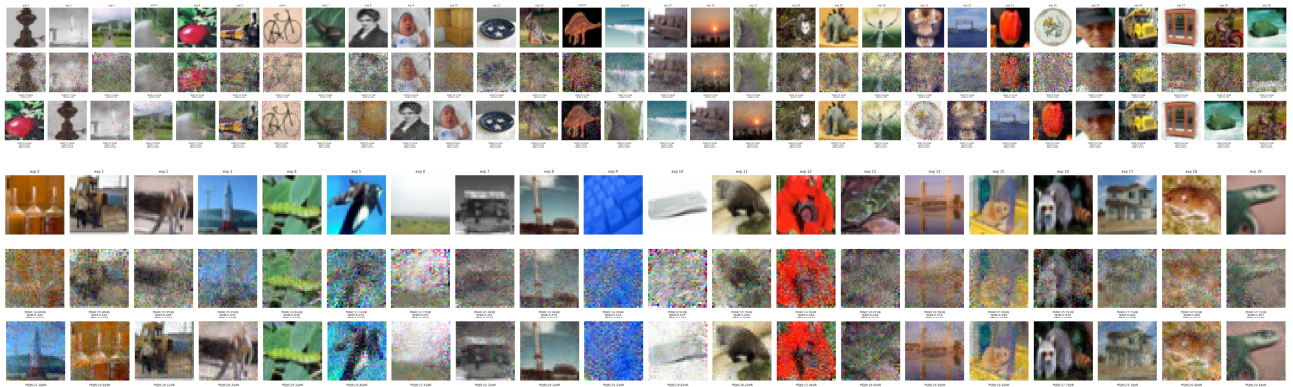
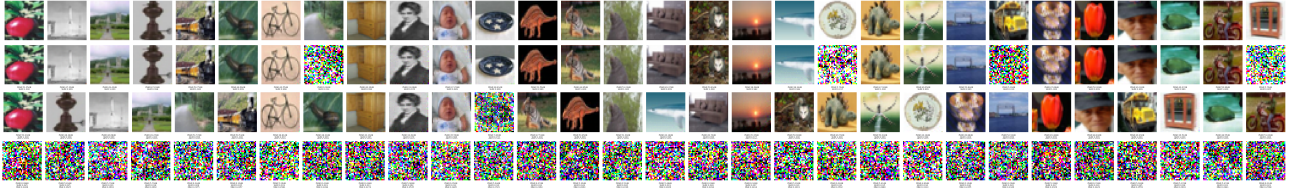


Figure D.3. Gradient inversion reconstructions for all 50 LFW and 50 CIFAR-100 samples using VGG-13. Within each dataset, the upper image of each pair shows reconstructions obtained from the full gradient, while the lower image shows reconstructions obtained using the best-performing masking configuration (no pretraining, Signed AdamW, cosine-similarity). Samples 1–30 and 31–50 are shown separately for readability.

D.3 LeNet

L-BFGS, L2 loss, 300 iterations



Signed AdamW, cosine-similarity, 5000 iterations

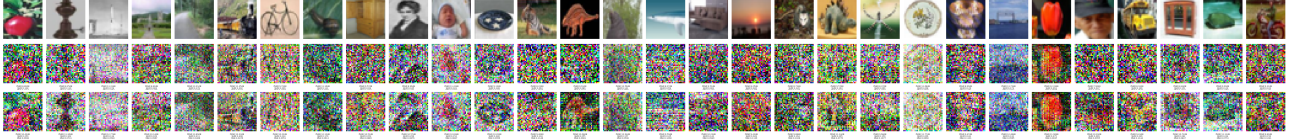


Figure D.4. Gradient inversion reconstructions for CIFAR-100 samples 1–30 using LeNet. The upper group corresponds to experiments optimized with L-BFGS and L2 loss for 300 iterations. From top to bottom, the rows show the ground-truth images, reconstructions obtained from the full gradient, reconstructions obtained with the classifier layer (`fc`) masked, and reconstructions obtained with the second convolutional block (`body . 2`) masked. The lower group corresponds to experiments optimized with Signed AdamW and cosine-similarity for 5000 iterations. From top to bottom, the rows show the ground-truth images, reconstructions obtained from the full gradient, and reconstructions obtained with the classifier layer (`fc`) masked.

D.4 ResNet

ResNet-18



ResNet-50

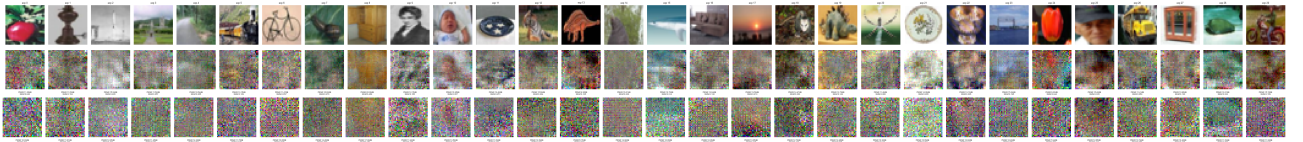


Figure D.5. Gradient inversion reconstructions for CIFAR-100 samples 1–30 using ResNet-18 and ResNet-50. Within each model, the top row shows the ground-truth images, the second row shows reconstructions obtained from the full gradient, and the third row shows reconstructions obtained after masking the `conv1` layer. All experiments use L-BFGS optimization with L2 loss for 300 iterations.

Model Parameter Counts

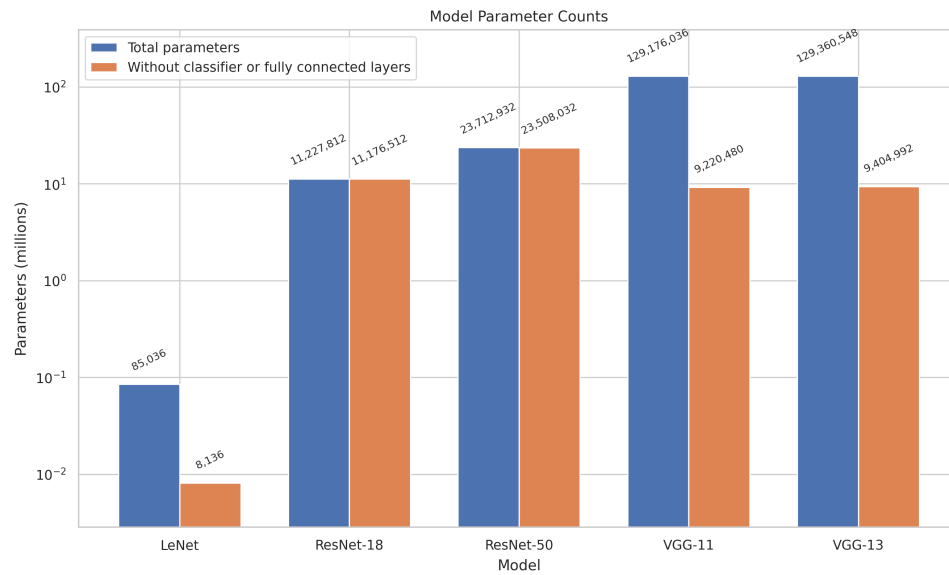


Figure E.1. Total parameter count (blue) and the count excluding the classifier and fully connected layers (orange), i.e. the convolutional backbone, for LeNet, ResNet-18, ResNet-50, VGG-11, and VGG-13 on a logarithmic scale. For the VGG models the two series diverge by more than an order of magnitude because the fully connected classifier holds most of the parameters (≈ 129 M total versus ≈ 9 M without it), whereas for the ResNet models, which use a single linear layer after global average pooling, the two counts nearly coincide.

DTU Compute

Technical University of Denmark
Department of Applied Mathematics and Computer Science

Building 324
2800 Kongens Lyngby
Tlf. 4525 1700

<https://www.compute.dtu.dk>